

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM
KHOA KỸ THUẬT MÁY TÍNH



BÁO CÁO ĐỒ ÁN
THIẾT KẾ HỆ THỐNG SỐ VỚI HDL
ĐỀ TÀI: SECURE HASH ALGORITHM (SHA 256)

Môn học: CE213.Q24 – Thiết kế hệ thống số với HDL

Giảng viên hướng dẫn: Ths. Ngô Hiếu Trường

Thực hiện bởi:

- | | |
|-----------------------|----------|
| 1. Lê Đăng Khôi | 23520765 |
| 2. Võ Nguyễn Anh Khôi | 23520791 |

Thời gian thực hiện: 3/2026 – 5/2026

LỜI NÓI ĐẦU

Trong bối cảnh công nghệ thông tin ngày càng phát triển, bảo mật dữ liệu đã trở thành một yêu cầu thiết yếu trong hầu hết các hệ thống số hiện đại. Từ các giao dịch tài chính trực tuyến, hệ thống xác thực danh tính, cho đến mạng lưới Blockchain — tất cả đều dựa vào các thuật toán mật mã để đảm bảo tính toàn vẹn và an toàn của thông tin. Trong số đó, SHA-256 là một trong những thuật toán băm được sử dụng phổ biến và tin cậy nhất hiện nay.

Nhận thấy tầm quan trọng của thuật toán này, nhóm đã quyết định chọn đề tài "**Thiết kế và hiện thực thuật toán SHA-256 bằng HDL**" làm chủ đề cho đồ án môn CE213. Mục tiêu của nhóm là hiểu rõ cách SHA-256 hoạt động ở mức phân cứng và hiện thực nó bằng ngôn ngữ Verilog, từ việc mô tả RTL cho đến kiểm thử bằng testbench với các vector chuẩn của NIST.

Trong quá trình thực hiện, nhóm đã học được nhiều điều không chỉ về bản thân thuật toán mà còn về tư duy thiết kế phần cứng — cách tổ chức datapath, xây dựng máy trạng thái FSM, và kiểm chứng hoạt động của mạch qua mô phỏng trên ModelSim và Quartus Prime. Đây là những kỹ năng thực tế mà nhóm cho rằng sẽ rất có ích trong công việc sau này.

Mặc dù đã cố gắng hoàn thành tốt nhất trong khả năng, do giới hạn về thời gian và kinh nghiệm, báo cáo chắc chắn vẫn còn nhiều điểm cần cải thiện. Nhóm rất mong nhận được nhận xét và góp ý từ thầy Ngô Hiếu Trường để có thể hoàn thiện hơn.

Nhóm em xin chân thành cảm ơn.

NHẬN XÉT CỦA GIẢNG VIÊN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

....., ngày.....tháng.....năm 2026

Người nhận xét

(Ký tên và ghi rõ họ tên)

MỤC LỤC

DANH MỤC HÌNH ẢNH.....	6
DANH MỤC BẢNG BIỂU.....	8
DANH MỤC KÝ TỰ VIẾT TẮT.....	9
THÔNG TIN THÀNH VIÊN THỰC HIỆN.....	10
Chương I. GIỚI THIỆU.....	11
1.1 Lý do chọn đề tài	11
1.2 Mục tiêu nghiên cứu	11
1.3 Đối tượng nghiên cứu	12
1.4 Phạm vi nghiên cứu	12
Chương II. TỔNG QUAN VỀ THUẬT TOÁN SHA-256.....	14
2.1 Giới thiệu chung về SHA-256	14
2.2 Kiến trúc tổng quát	14
Chương III. PHÂN TÍCH VÀ THIẾT KẾ KIẾN TRÚC	17
3.1 Sơ đồ khối tổng quát.....	17
3.2 Phân tích thuật toán	17
3.2.1 Khối message.....	17
3.2.2 Khối compression	18
3.3 Thiết kế FSM	21
3.3.1 FSM Khối Message	21
3.3.2 FSM Khối Compression	22
3.4 Kiến trúc	22
Chương IV. HIỆN THỰC THUẬT TOÁN	24
4.1 Khối tạo hằng số K	24
4.2 Khối Message	25
4.2.1 Khai báo Input/Output và thanh ghi	25
4.2.2 Các phép toán	26
4.2.3 Kỹ thuật ánh xạ địa chỉ tuần hoàn	26
4.2.4 Khối điều khiển (Finite State Machine – FSM)	28
4.3 Khối Compression	30
4.3.1 Khai báo Input/Output và thanh ghi	30

4.3.2 Các phép toán	31
4.3.3 Luồng tính toán 2 tầng (Two-Stage Computational Path).....	32
4.3.4 Khối điều khiển (Finite State Machine – FSM)	33
4.4 SHA_Top Module.....	35
Chương V. MÔ PHỎNG VÀ KẾT QUẢ.....	38
5.1 Thiết lập môi trường và phương pháp kiểm chứng	38
5.2 Mô phỏng Pre-Simulation	38
5.2.1 Kịch bản mô phỏng	40
5.2.2 Kết quả mô phỏng Pre-Simulation	42
5.3 Mô phỏng Post-Simulation.....	43
5.4 Tổng hợp phần cứng và đánh giá tài nguyên.....	45
5.5 Kết luận và hướng phát triển trong tương lai	48
NGUỒN THAM KHẢO	49

DANH MỤC HÌNH ẢNH

Hình 2.1.1: Bảng so sánh các thuật toán băm thuộc họ SHA-2	14
Hình 2.2.1: Mô hình Merkle-Damgård.....	15
Hình 2.2.2: Kiến trúc tổng quát SHA - 256.....	15
Hình 3.1: Sơ đồ khối tổng quát SHA – 256.....	17
Hình 3.2.1: Kiến trúc khối message	18
Hình 3.2.2.1: Giá trị khởi tạo của 8 thanh ghi trạng thái.....	18
Hình 3.2.2.2: Quy trình thực hiện nén.....	20
Hình 3.3.1: Sơ đồ chuyển trạng thái khối Message.....	21
Hình 3.3.2: Sơ đồ chuyển trạng thái khối Compression.....	22
Hình 4.1: Khối tạo hằng số K.....	24
Hình 4.2.1.1: Khai báo I/O và thanh ghi cho khối Message	25
Hình 4.2.2: Các phép toán trong khối Message.....	26
Hình 4.2.4: Các trạng thái của Message	28
Hình 4.3.1: Khai báo I/O và thanh ghi khối Compression	30
Hình 4.3.2: Các phép toán khối Compression.....	31
Hình 4.3.3: Tính toán 2 tầng.....	32
Hình 4.3.4: Các trạng thái của Compression.....	34
Hình 4.4: Khối SHA_Top Module	36
Hình 5.1.1: Testbench thực hiện mô phỏng Pre-Simulation	39
Hình 5.2.1.1: Các kịch bản ví dụ mô phỏng Pre-Simulation cho trường hợp đơn giản và biên.....	40
Hình 5.2.1.2: Các kịch bản ví dụ mô phỏng Pre-Simulation cho trường hợp 1 block.....	41
Hình 5.2.1.3: Các kịch bản ví dụ mô phỏng Pre-Simulation cho trường hợp chuỗi tiếng Việt có dấu	41
Hình 5.2.1.4: Các kịch bản ví dụ mô phỏng Pre-Simulation cho trường hợp nhiều hơn 1 block	42
Hình 5.2.2.1: Kết quả mô phỏng dạng sóng Pre-Simulation.....	42
Hình 5.2.2.2: Kết quả mô phỏng Pre-Simulation hiển thị trên transcript.....	43
Hình 5.3.1: Kết quả mô phỏng dạng sóng Post-Simulation	44
Hình 5.3.2: Kết quả mô phỏng Pre-Simulation hiển thị trên transcript.....	45

Báo cáo Đồ án Thiết kế hệ thống số với HDL

Hình 5.4.1: Báo cáo tài nguyên sử dụng trên kit ALTERA DE2.....	46
Hình 5.4.2: Công suất tiêu thụ thực tế.....	47

DANH MỤC BẢNG BIỂU

Bảng 4.4.1: Các chân tín hiệu giao tiếp của khối top_sha256.....	36
Bảng 4.4.2: Tín hiệu định tuyến nội bộ của khối sha_top	37
Bảng 5.4.1: Bảng phân tích tài nguyên sử dụng.....	46
Bảng 5.4.2: Bảng công suất tiêu thụ.....	47

DANH MỤC KÝ TỰ VIẾT TẮT

Tên viết tắt	Mô tả
SHA	Secure Hash Algorithm
FSM	Finite State Machine
ROTR	Rotate Right
SHR	Shift Right
Ch	Choose
Maj	Majority
FPGA	Field-Programmable Gate Array
NIST	National Institute of Standards and Technology
RTL	Register-Transfer Level
AXI	Advanced Extensible Interface
I/O	Input / Output
SoC	System-on-Chip
IP	Intellectual Property
IC	Integrated Circuit
FIPS	Federal Information Processing Standards
IV	Initialization Vector
TLS/SSL	Transport Layer Security/ Secure Sockets Layer

THÔNG TIN THÀNH VIÊN THỰC HIỆN

Lớp: CE213.Q24

STT	Họ và tên	MSSV
1	Lê Đăng Khôi	23520765
2	Võ Nguyễn Anh Khôi	23520791

Chương I. GIỚI THIỆU

1.1 Lý do chọn đề tài

Sự bùng nổ của kỷ nguyên số và Internet vạn vật (IoT) đã dẫn đến sự gia tăng theo cấp số nhân của các luồng dữ liệu được truyền tải và lưu trữ mỗi ngày. Kéo theo đó, bài toán bảo mật, tính toàn vẹn và khả năng xác thực thông tin trở thành yêu cầu sống còn đối với các nền tảng công nghệ lõi như tài chính số, thương mại điện tử, điện toán đám mây và hệ thống chuỗi khối (Blockchain). Trong bức tranh toàn cảnh đó, các hàm băm mật mã (Cryptographic Hash Functions) được xem là lớp khiên bảo vệ vững chắc và không thể thiếu.

Tiêu biểu và được ứng dụng rộng rãi nhất hiện nay là SHA-256, chuẩn băm mật mã thuộc họ SHA-2 do NIST ban hành. Với đầu ra là dải bit cố định có chiều dài 256-bit, thuật toán này sở hữu khả năng kháng va chạm (Collision Resistance) ưu việt và miễn nhiễm với các kỹ thuật tấn công dò tìm. Ngày nay, SHA-256 đã trở thành hạt nhân trung tâm trong cấu trúc của các giao thức mã hóa mạng (TLS/SSL), hệ thống chứng thực chữ ký số, và là nền tảng duy trì tính bất biến của các sổ cái phân tán.

Tuy nhiên, khi đối mặt với yêu cầu khắt khe về băng thông dữ liệu khổng lồ và độ trễ cực thấp trong các hệ thống thời gian thực, các kiến trúc vi xử lý trung tâm truyền thống bắt đầu bộc lộ những điểm nghẽn nghiêm trọng về tốc độ xử lý cũng như năng lượng tiêu thụ. Điều này đòi hỏi sự ra đời của các lõi vi mạch gia tốc phần cứng (Hardware Accelerator IP Core) chuyên dụng. Việc chuyển dịch và hiện thực hóa thuật toán SHA-256 trên các nền tảng thiết kế vi mạch (như FPGA hoặc ASIC) cho phép khai thác triệt để sức mạnh tính toán song song ở mức cổng logic. Qua đó, phần cứng không chỉ đẩy thông lượng (Throughput) lên mức tối đa, giảm thiểu độ trễ, mà còn đáp ứng được các tiêu chuẩn khắt khe về công suất tiêu thụ (Low Power) và tối ưu tài nguyên hệ thống.

Nhận thức rõ xu hướng và tầm quan trọng của lĩnh vực vi mạch bảo mật, nhóm đã quyết định thực hiện đề tài **“Thiết kế và hiện thực thuật toán SHA-256 bằng HDL”**. Quá trình thực hiện đồ án không chỉ giúp nhóm đào sâu vào vi kiến trúc và luồng dữ liệu của chuẩn mã hóa quốc tế, mà còn là cơ hội thực tiễn để rèn luyện tư duy thiết kế luận lý số. Thông qua các công đoạn từ phân tích yêu cầu, mô tả RTL cho đến kiểm chứng mô phỏng, nhóm kỳ vọng sẽ củng cố vững chắc các kỹ năng thiết kế phần cứng chuyên sâu, từng bước tiếp cận với các tiêu chuẩn và quy trình làm việc trong môi trường công nghiệp vi mạch.

1.2 Mục tiêu nghiên cứu

Đồ án hướng tới việc xây dựng một hệ thống phần cứng thực thi thuật toán SHA-256 hoàn chỉnh với các mục tiêu cụ thể như sau:

- Mô hình hóa chính xác thuật toán SHA-256 theo chuẩn FIPS 180-4 ở mức thanh ghi chuyển (RTL) bằng ngôn ngữ mô tả phần cứng Verilog.
- **Tối ưu hóa tốc độ (Throughput):** Cải tiến vi kiến trúc cơ bản bằng kỹ thuật 2-Stage Unrolling (Gộp 2 vòng tính toán) kết hợp với cây cộng song song (Adder Tree) nhằm giảm đi một nửa số lượng chu kỳ xung nhịp (Clock Cycles) cần thiết để hoàn thành bản băm.
- **Tối ưu hóa tài nguyên (Area):** Áp dụng kỹ thuật bộ đệm vòng (Circular Buffer) cho khối mở rộng thông điệp (Message Schedule) nhằm giảm thiểu tối đa số lượng thanh ghi (Registers) phải sử dụng.
- Thiết lập môi trường kiểm chứng (Testbench) mạnh mẽ, thực hiện mô phỏng toàn diện (Pre-Simulation) để đối chiếu tính đúng đắn của đường truyền dữ liệu (Datapath) và mô phỏng (Post – Simulation) để xác định tài nguyên sử dụng.

1.3 Đối tượng nghiên cứu

Đồ án tập trung nghiên cứu sâu vào các khía cạnh cốt lõi của thiết kế vi mạch số, bao gồm:

- **Tiêu chuẩn mật mã:** Phân tích luồng dữ liệu (Datapath) và các phép toán logic của thuật toán băm SHA-256 theo đúng tài liệu chuẩn FIPS 180-4 do NIST ban hành.
- **Phương pháp thiết kế:** Ngôn ngữ mô tả phần cứng Verilog HDL và phương pháp luận thiết kế mạch tích hợp ở mức thanh ghi chuyển (RTL - Register Transfer Level).
- **Kỹ thuật tối ưu hóa vi kiến trúc (Microarchitecture Optimization):** Nghiên cứu và áp dụng các cấu trúc phần cứng như kiến trúc gộp vòng (2-Stage Unrolling), cây cộng song song (Adder Tree) để tối ưu hóa tốc độ; và bộ đệm vòng (Circular Buffer) để tối ưu hóa diện tích.
- **Phương pháp kiểm chứng (Verification):** Xây dựng môi trường thử nghiệm (Testbench) và so sánh với các Vector chuẩn.

1.4 Phạm vi nghiên cứu

Để đảm bảo tính chuyên sâu và hoàn thiện của đề tài trong khuôn khổ đồ án môn học, phạm vi nghiên cứu được giới hạn và xác định rõ ràng như sau:

- **Về mặt thiết kế hệ thống:** Đồ án tập trung vào việc thiết kế lõi IP SHA-256 dưới dạng một module gia tốc độc lập (Standalone Hardware Accelerator). Đồ án chưa bao gồm việc bao bọc (wrapper) lõi IP này bằng các chuẩn giao thức giao tiếp hệ thống (như AMBA AXI hay APB) để tích hợp vào một Hệ thống trên chip (SoC) hoàn chỉnh.
- **Về mặt kiểm chứng và đánh giá:** Đồ án thực hiện trọn vẹn hai công đoạn kiểm chứng quan trọng của luồng thiết kế phần cứng:
 - + *Mô phỏng trước tổng hợp (Pre-Simulation / RTL Simulation):* Đưa toàn bộ mã nguồn Verilog vào môi trường lý tưởng để kiểm chứng tính đúng đắn tuyệt đối của

thuật toán và máy trạng thái (FSM), đảm bảo hệ thống bấm ra kết quả chính xác 100% so với lý thuyết.

+ *Mô phỏng sau tổng hợp (Post-Simulation / Gate-level Simulation)*: Ánh xạ mã nguồn thành sơ đồ cổng logic vật lý thực tế (Netlist) có tính đến các độ trễ vật lý của cổng logic (Gate Delay) và độ trễ đường dây (Routing Delay). Từ đó, kiểm tra các vi phạm về thời gian và chứng minh hệ thống có khả năng hoạt động ổn định ở tần số xung nhịp mục tiêu.

- **Về mặt công cụ (EDA Tools)**: Đồ án giới hạn việc sử dụng công cụ tổng hợp phần cứng Intel Quartus Prime để trích xuất Netlist/độ trễ tĩnh, và phần mềm ModelSim/QuestaSim để phân tích dạng sóng mô phỏng.

Chương II. TỔNG QUAN VỀ THUẬT TOÁN SHA-256

2.1 Giới thiệu chung về SHA-256

SHA-2 (Secure Hash Algorithm 2) là thể hệ hàm băm mật mã an toàn do Cơ quan An ninh Quốc gia Hoa Kỳ (NSA) thiết kế và được công bố rộng rãi vào năm 2001. Kế thừa và phát triển từ các thể hệ trước, kiến trúc của SHA-2 được định hình dựa trên nguyên lý Merkle-Damgård tích hợp cùng mạng lưới hàm nén một chiều phức tạp. Nhằm đáp ứng đa dạng các cấp độ bảo mật, họ SHA-2 được phân nhánh thành nhiều chuẩn độ dài khác nhau, bao gồm: SHA-224, SHA-256, SHA-384, SHA-512, SHA-512/224 và SHA-512/256.

Parameter	SHA-256	SHA-512	SHA-224	SHA-384	SHA-512/224	SHA-512/256
Hash size (bit)	256	512	224	384	224	256
Block size (bit)	512	1024	512	1024	1024	1024
Message size (bit)	$< 2^{64}$	$< 2^{128}$	$< 2^{64}$	$< 2^{128}$	$< 2^{128}$	$< 2^{128}$
Security bits	128	256	112	192	112	128
Corresponding symmetric cipher	AES-128	AES-256	TDES	AES-192	TDES	AES-128
Word size (bit)	32	64	32	64	64	64
Number of rounds	64	80	64	80	80	80
Publication year	2001	2001	2004	2001	2012	2012

Hình 2.1.1: Bảng so sánh các thuật toán băm thuộc họ SHA-2

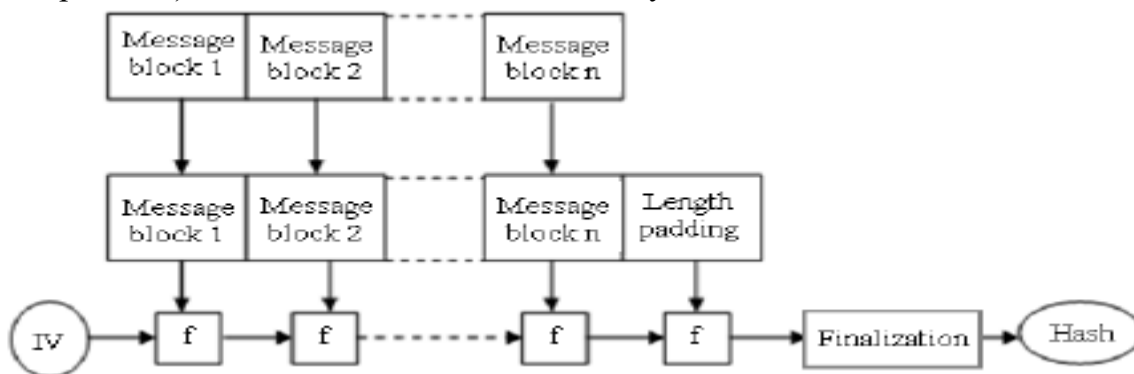
Nổi bật và được ứng dụng rộng rãi nhất trong họ thuật toán này là SHA-256 (Secure Hash Algorithm 256-bit) – chuẩn băm mật mã một chiều (One-way Hash Function) đã được Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST) quy chuẩn hóa tại tài liệu FIPS 180-4. Về bản chất toán học, SHA-256 là một quá trình ánh xạ dữ liệu nghiêm ngặt: nó có khả năng tiếp nhận một thông điệp thô đầu vào với kích thước biến thiên (lên đến $2^{64} - 1$ bit) và nén gọn thành một bản tóm tắt thông điệp (Message Digest) có độ dài cố định chính xác 256 bit.

Sức mạnh cốt lõi của SHA-256 nằm ở "Hiệu ứng thác" (Avalanche Effect) và "Khả năng kháng va chạm" (Collision Resistance). Hiệu ứng thác đảm bảo rằng sự thay đổi của chỉ một bit duy nhất ở thông điệp đầu vào sẽ gây ra hiệu ứng dây chuyền qua mạng lưới các phép toán phi tuyến, làm thay đổi hoàn toàn (xác suất khoảng 50%) các bit ở kết quả đầu ra. Điều này khiến cho việc dịch ngược từ mã băm ra dữ liệu gốc, hoặc việc cố tình nguy tạo hai thông điệp khác nhau cho ra cùng một mã băm, là điều bất khả thi với năng lực tính toán hiện tại.

2.2 Kiến trúc tổng quát

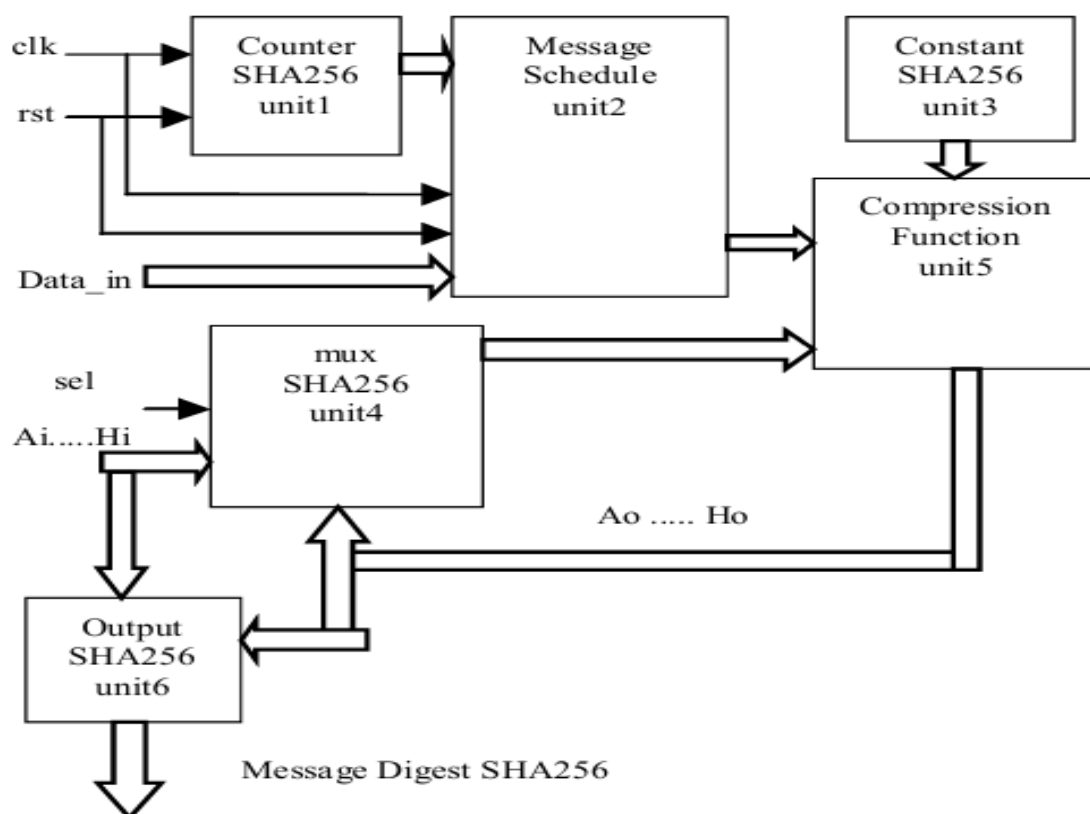
Về mặt kiến trúc tổng thể, SHA-256 được xây dựng trên nền tảng của mô hình Merkle-Damgård. Cơ chế cốt lõi của mô hình này là phân rã luồng dữ liệu thành các khối (blocks) độc lập và đưa qua một mạng lưới hàm nén lặp đi lặp lại (iterative

compression) để đúc kết thành một mã băm duy nhất.



Hình 2.2.1: Mô hình Merkle-Damgård

Kiến trúc tổng quát của thuật toán SHA – 256:



Hình 2.2.2: Kiến trúc tổng quát SHA - 256

Giai đoạn 1: Tiền xử lý và Nhồi bit (Preprocessing & Padding)

Trong thực tế, dữ liệu đầu vào hiếm khi có chiều dài chia hết cho 512 bit. Để hệ thống có thể xử lý, ta phải đệm thêm bit để dữ liệu đạt chuẩn này. Đầu tiên, một bit 1 được nối vào cuối dữ liệu, theo sau là một dải các bit 0. Việc thêm bit 0 sẽ dừng lại khi chuỗi dữ liệu còn cách mốc 512 bit đúng 64 bit. Khoảng trống 64-bit cuối cùng này được dùng để lưu lại con số thể hiện chiều dài ban đầu của thông điệp. Việc này giúp ngăn chặn kẻ gian cố tình nối ghép thêm dữ liệu giả mạo vào đuôi gói tin.

Giai đoạn 2: Tạo mảng thông điệp (Message Schedule)

Hệ thống không xử lý toàn bộ khối 512-bit cùng một lúc mà chặt nó ra thành 16 phần nhỏ, mỗi phần rộng 32-bit (gọi là các từ W_0 đến W_{15}). Từ 16 từ cơ sở này, mạch logic sẽ dùng các phép xoay và dịch bit (σ_0, σ_1) để tính toán và sinh ra thêm 48 từ mới. Kết quả là ta thu được một danh sách gồm 64 từ (W_0 đến W_{63}). Bước này giúp xáo trộn thông tin, xóa bỏ mọi quy luật hay thói quen lặp lại của dữ liệu gốc.

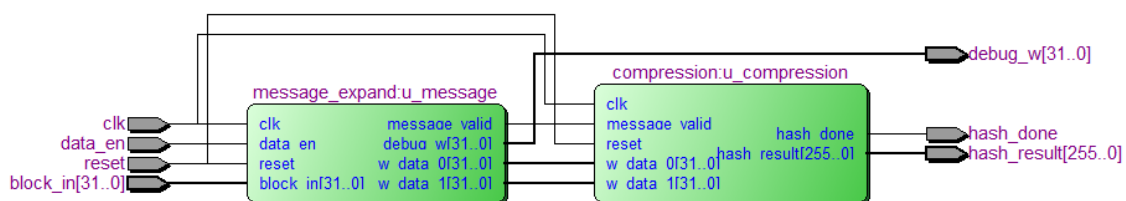
Giai đoạn 3: Vòng lặp nén (Compression Rounds)

Đây là bước tính toán cốt lõi, hoạt động dựa trên 8 thanh ghi trạng thái (từ A - H). Quá trình nén được lặp đi lặp lại 64 lần (64 vòng). Tại mỗi vòng lặp, hệ thống lấy một từ dữ liệu W_t và một hằng số K_t đưa qua các cổng logic phức tạp (như Maj, Ch, Σ_0, Σ_1). Kết quả của các cổng này được dùng để cập nhật lại giá trị cho 8 thanh ghi. Một quy tắc quan trọng là mọi phép cộng ở bước này đều là cộng Modulo 2^{32} (nghĩa là nếu cộng vượt quá 32 bit thì phần dư sẽ tự động bị vớt bỏ). Khi kết thúc 64 vòng, giá trị của 8 thanh ghi sẽ được cộng dồn với kết quả của khối trước đó. Chuỗi 256-bit thu được ở cuối quy trình chính là mã băm hoàn chỉnh.

Chương III. PHÂN TÍCH VÀ THIẾT KẾ KIẾN TRÚC

3.1 Sơ đồ khối tổng quát

Như đã phân tích ở trên thì hệ thống được xây dựng xử lý song song gồm 2 khối chức năng chính là: Message, Compression. Mỗi khối sẽ có chức năng riêng và vai trò cụ thể nhưng chúng sẽ phối hợp với nhau theo đúng quy trình của thuật toán SHA – 256.



Hình 3.1: Sơ đồ khối tổng quát SHA – 256

3.2 Phân tích thuật toán

3.2.1 Khối message

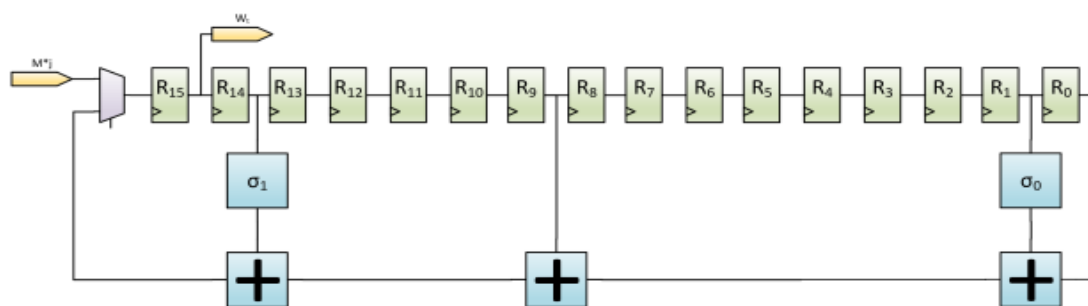
Khối Mở rộng thông điệp (message_expand) đóng vai trò là bộ tiền xử lý luồng dữ liệu cục bộ. Nhiệm vụ của khối này là tiếp nhận khối dữ liệu 512-bit (được chia thành 16 từ 32-bit cơ sở, từ W_0 đến W_{15} và thực hiện quá trình nội suy để mở rộng thành dài 64 từ 32-bit W_0 đến W_{63} .

Quá trình nội suy từ bước thứ 16 trở đi được thực hiện thông qua công thức toán học:

$$W_t = \sigma_1(W_{t-2}) + W_{t-7} + \sigma_0(W_{t-15}) + W_{t-16}$$

Trong đó, σ_0 và σ_1 là các hàm xoay và dịch bit phi tuyến:

- $\sigma_0(x) = \text{ROTR}(x, 7) \wedge \text{ROTR}(x, 18) \wedge \text{SHR}(x, 3)$
- $\sigma_1(x) = \text{ROTR}(x, 17) \wedge \text{ROTR}(x, 19) \wedge \text{SHR}(x, 10)$



Hình 3.2.1: Kiến trúc khối message

Thông thường, khối Message chỉ tính toán và xuất ra 1 từ W_t mỗi chu kỳ xung nhịp. Tuy nhiên, do khối Hàm nén của hệ thống được thiết kế lại theo kiến trúc gộp vòng 2 mức (2-Stage Unrolling), nó đòi hỏi phải được cung cấp hai từ dữ liệu đồng thời trong một chu kỳ.

Do đó, mạch tổ hợp bên trong khối message_expand đã được thiết kế lại để thực hiện tính toán song song. Tại bất kỳ chu kỳ xung nhịp nào, hệ thống tính toán và xuất ra cùng lúc hai tín hiệu: w_data_0 (đại diện cho W_t) và w_data_1 (đại diện cho W_{t+1}). Để tối ưu hóa tài nguyên phần cứng (Area), hệ thống sử dụng một bộ đệm vòng (Circular Buffer) mảng w 16 thanh ghi hoạt động theo cơ chế cuốn chiếu, kết hợp với tín hiệu reset tích cực mức thấp (reset active-low) để đảm bảo trạng thái an toàn trước khi nạp khối dữ liệu mới.

3.2.2 Khối compression

Khối Hàm nén (compression) là khối quản lý 8 thanh ghi trạng thái (từ a đến h), cập nhật giá trị của chúng liên tục qua 64 vòng lặp. Các thanh ghi trạng thái này được khởi tạo bằng 32 bit phân số của căn bậc 2 của 8 số nguyên tố đầu tiên (2 – 19):

$$a = 0x6a09e667$$

$$b = 0xbb67ae85$$

$$c = 0x3c6ef372$$

$$d = 0xa54ff53a$$

$$e = 0x510e527f$$

$$f = 0x9b05688c$$

$$g = 0x1f83d9ab$$

$$h = 0x5be0cd19$$

Hình 3.2.2.1: Giá trị khởi tạo của 8 thanh ghi trạng thái

Với thiết kế của Compression được chia làm 2 tầng thể hiện bản chất của kỹ thuật Two Stage Unrolling (Gỡ vòng 2 bước):

Tầng 1 (vòng t):

Mạch tổ hợp sử dụng trạng thái hiện tại $a - h$, kết hợp với toán hạng thứ nhất w_data_0 và k_val_0 . Kết quả sinh ra không được chốt vào thanh ghi mà gán cho hai tín hiệu dây (wire) là e_mid và a_mid .

$$T_1 = h + \Sigma_1(E) + Ch(E, F, G) + (K_{[t]} + W_{[t]})$$

$$T_2 = \Sigma_0(A) + Maj(A, B, C)$$

$$e_mid = d + T_1$$

$$a_mid = t_1 + t_2$$

Tầng 2 (Vòng t+1):

Logic tổ hợp của tầng 2 lấy e_mid và a_mid từ tầng 1. Nó kết hợp với toán hạng thứ hai w_data_1 , k_val_1 để tính toán ra biến $t1_next$ và $t2_next$. Cuối cùng, hệ thống sinh ra trạng thái đích cho chu kỳ tiếp theo:

$$a_next = T1_next + T2_next;$$

$$e_next = c + T1$$

Các công thức bên trong bao gồm:

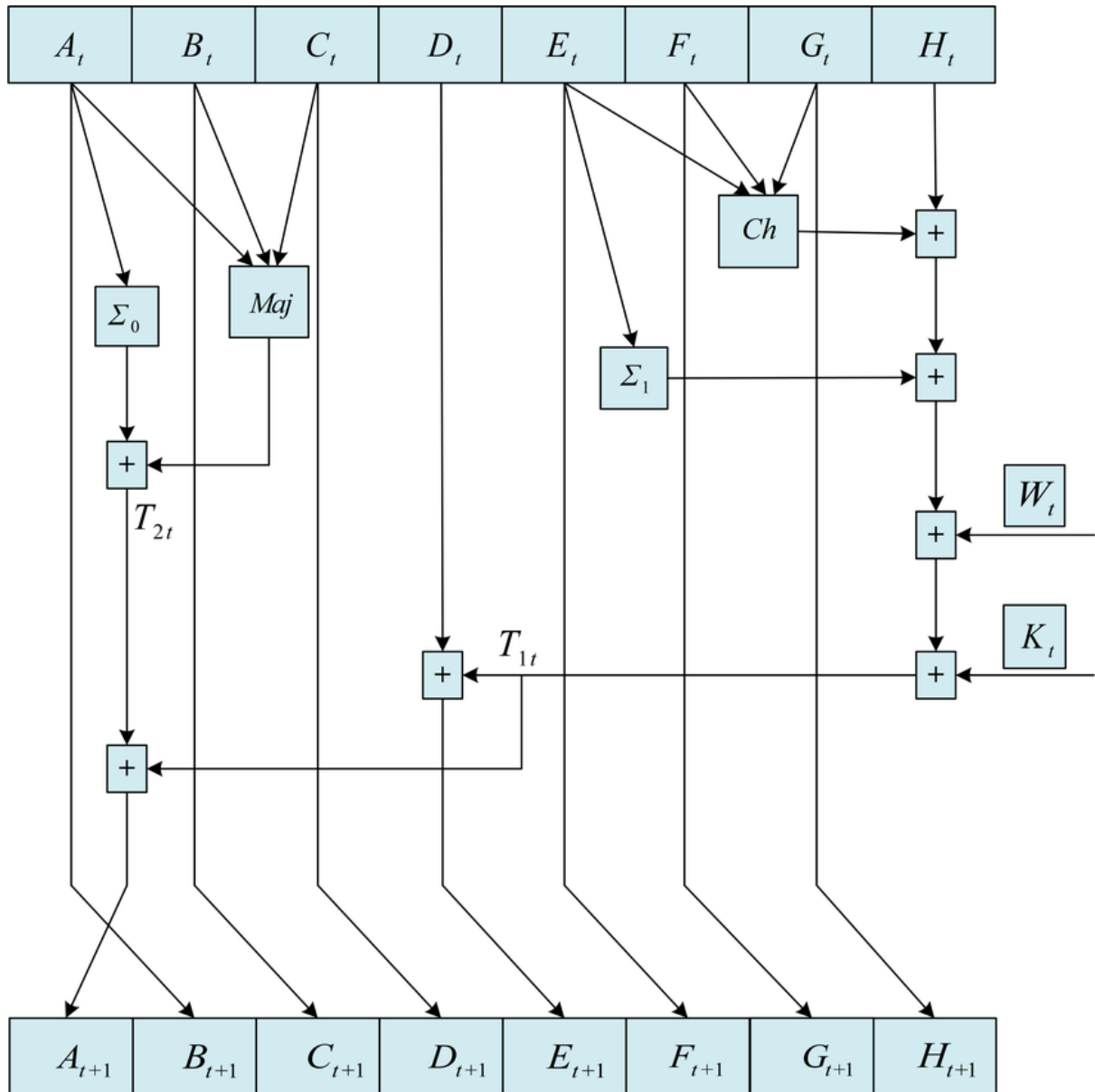
- $Ch(e, f, g) = (e \& f) \wedge (\sim e \& g)$
- $Maj(a, b, c) = (a \& b) \wedge (a \& c) \wedge (b \& c)$
- $\Sigma_0(a) = ROTR(a, 2) \wedge ROTR(a, 13) \wedge ROTR(a, 22)$
- $\Sigma_1(e) = ROTR(e, 6) \wedge ROTR(e, 11) \wedge ROTR(e, 25)$

Cơ chế dịch nhảy bậc (Jump Shifting):

Do hệ thống thực hiện 2 vòng lặp vật lý trong 1 chu kỳ xung nhịp, quy luật dịch thanh ghi thông thường ($h \leftarrow g \leftarrow f \dots$) không còn đúng. Để kiến trúc Unrolling hoạt động, việc dịch thanh ghi phải bước nhảy 2 bậc:

- $h \leftarrow f$ (Thanh ghi h nhận giá trị của f từ 2 vòng trước)

- $g \leftarrow e$
- $f \leftarrow e_mid$ (Thanh ghi f lưu giá trị e trung gian của vòng 1)
- $e \leftarrow e_next$ (Thanh ghi e cập nhật kết quả cuối của vòng 2)
- $d \leftarrow b$
- $c \leftarrow a$
- $b \leftarrow a_mid$ (Thanh ghi b lưu giá trị a trung gian của vòng 1)
- $a \leftarrow a_next$ (Thanh ghi a cập nhật kết quả cuối của vòng 2)



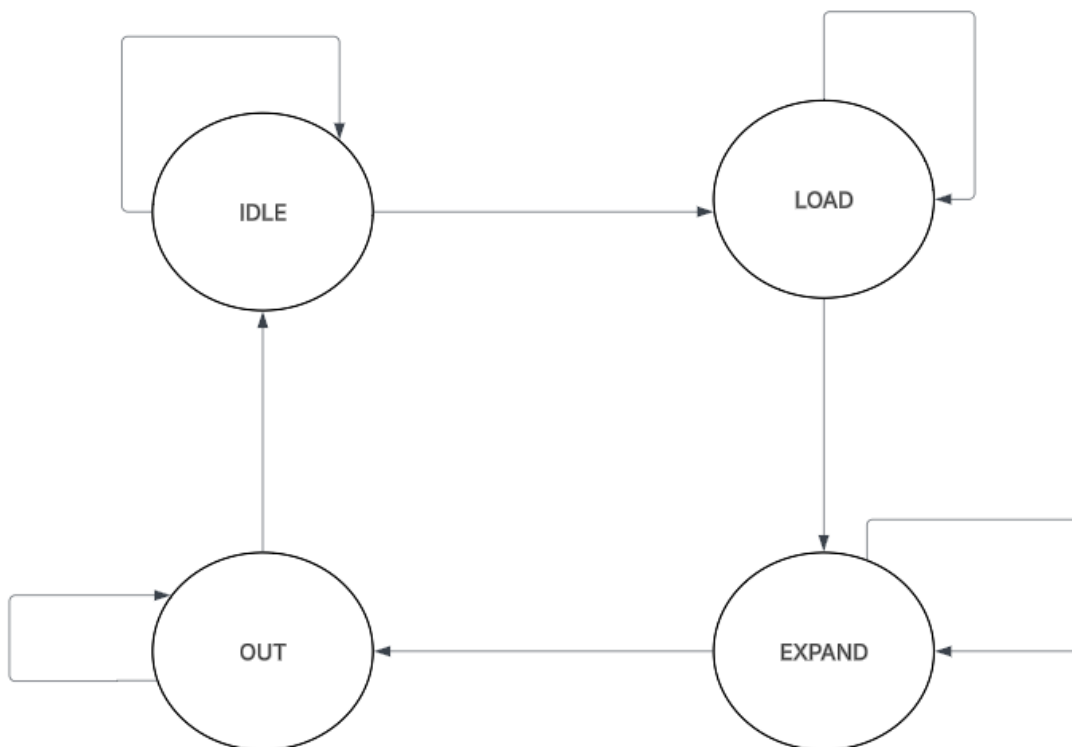
Hình 3.2.2.2: Quy trình thực hiện nén

Với thuật toán SHA – 256 chuẩn thì khối Compression phải thực hiện các vòng nén trong 64 chu kỳ clock nhưng vì đồ án sử dụng kiến trúc gỡ vòng 2 bước (Two Stage Unrolling) nên khối Compression sẽ hoàn thành trong 32 chu kỳ clock.

3.3 Thiết kế FSM

3.3.1 FSM Khối Message

Hoạt động nạp và nội suy dữ liệu được kiểm soát bởi FSM 4 trạng thái:

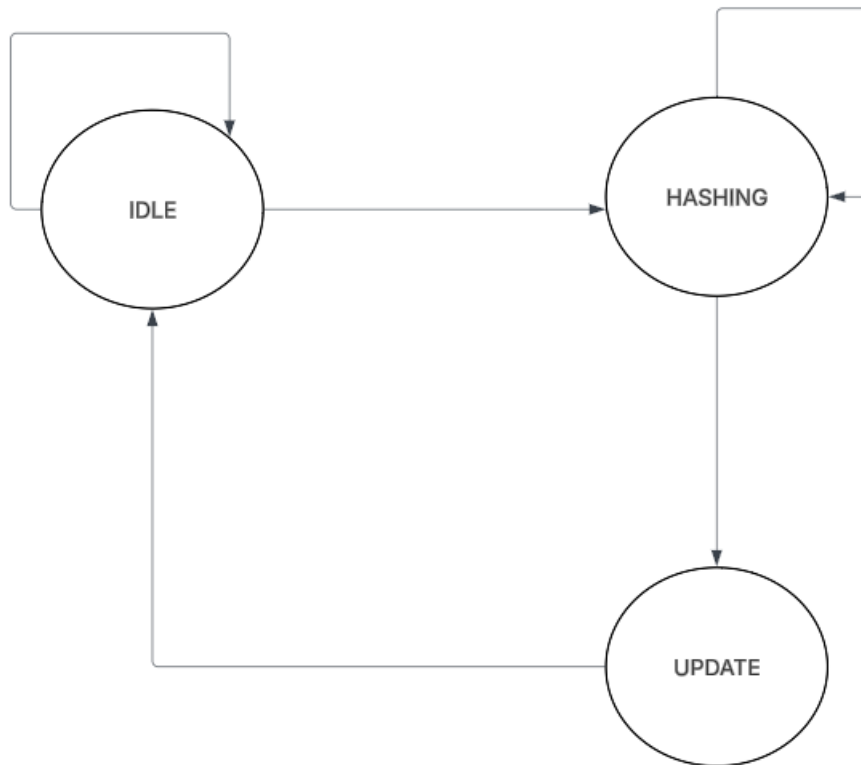


Hình 3.3.1: Sơ đồ chuyển trạng thái khối Message

- **IDLE:** Khối chờ tín hiệu `data_en = 1` để khởi động quá trình nạp.
- **LOAD:** Mạch tuần tự nạp từng từ 32-bit từ `block_in` vào bộ đệm vòng `w` thông qua biến trỏ `load_idx`. Khi `load_idx = 15` (đã nạp đủ 16 từ), mạch kích hoạt `valid_out = 1` và chuyển sang trạng thái tính toán.
- **EXPAND:** Đây là trạng thái tính toán. Biến đếm `word_cnt` bắt đầu từ 0 và tăng 2 đơn vị sau mỗi chu kỳ xung nhịp ($word_cnt \leq word_cnt + 2$) do mạch phải cung cấp 2 từ thông điệp cùng lúc. Trạng thái này duy trì việc nội suy các giá trị mới cho đến khi `word_cnt == 46`.
- **OUT:** Giai đoạn vét cạn cuối cùng. Từ `word_cnt = 48` đến 62, khối dừng ghi dữ liệu mới vào bộ đệm, chỉ xuất các giá trị đã tồn tại ra cổng `w_data_0` và `w_data_1`. Sau khi đạt 62, FSM quay về IDLE.

3.3.2 FSM Khối Compression

Khối nén vận hành dựa trên FSM 3 trạng thái:



Hình 3.3.2: Sơ đồ chuyển trạng thái khối Compression

- **IDLE:** Trạng thái nghỉ, duy trì hash_done = 0 và bộ đếm count = 0. Khi nhận được message_valid = 1 từ khối Message, nó nạp các giá trị khởi tạo (hoặc Hash trung gian từ vòng trước) hash_0... hash_7 vào 8 thanh ghi làm việc a...h.
- **HASING:** Trạng thái nén lõi. Nhờ kiến trúc gỡ vòng, bộ đếm count chỉ cần chạy từ 0 đến 31 (thay vì 63). Mỗi nhịp clock mạch hoàn tất 2 vòng nén. Khi count == 31, FSM chuyển sang pha xuất dữ liệu.
- **UPDATE:** Bật cờ hoàn thành hash_done = 1. Tại đây, hệ thống thực hiện phép cộng dồn Modulo 2^{32} giữa trạng thái cuối cùng của 8 thanh ghi với các giá trị Hash cũ để chốt kết quả, sẵn sàng cho khối dữ liệu tiếp theo.

3.4 Kiến trúc

Như đã phân tích, kiến trúc vi mạch trong đồ án là sự kết hợp chéo giữa kỹ thuật Gỡ vòng 2 bước (Two-Stage Unrolling) và Bộ đệm vòng (Circular Buffer). Điểm đột phá của kiến trúc này là khả năng hoàn tất 64 vòng nén phi tuyến chỉ trong 32 chu kỳ xung nhịp, qua đó đẩy thông lượng (Throughput) của hệ thống lên gấp đôi so với các mô hình phần cứng tuần tự truyền thống.

Tuy nhiên, sự vượt trội về mặt tốc độ luôn đi kèm với bài toán đánh đổi tài nguyên. Để có thể tính toán gộp hai vòng nén trong một chu kỳ đồng hồ, vi kiến trúc của khối Compression buộc phải mở rộng các trạm logic tổ hợp. Kéo theo đó, mạng lưới định tuyến từ khối Message và bộ nhớ ROM cũng phải nâng cấp bằng thông để đáp ứng khả năng cấp phát đồng thời các cặp toán w_data_0 , w_data_1 và k_val_0 , k_val_1 . Hệ quả tất yếu là diện tích vi mạch (Silicon Area) và số lượng phần tử logic sẽ tăng lên so với thuật toán gốc. Dù vậy, đối với mục tiêu thiết kế một bộ gia tốc phần cứng chuyên dụng, sự đánh đổi diện tích để lấy hiệu năng này là hoàn toàn có chủ đích và xứng đáng.

Chương IV. HIỆN THỰC THUẬT TOÁN

4.1 Khối tạo hằng số K

SHA-256 sử dụng 64 hằng số 32-bit ($K_0, K_1, \dots, K_{63}$), được định nghĩa là 32-bit đầu tiên của phần phân số trong căn bậc ba của 64 số nguyên tố đầu tiên (từ 2 đến 311). Các giá trị này cố định, được biểu diễn dưới dạng thập lục phân (hexadecimal) để đảm bảo tính minh bạch và chống backdoor.

```
module constant_k (
    input      [4:0] count,
    output reg [31:0] k_val_0,
    output reg [31:0] k_val_1
);
    always @(*) begin
        case (count)
            0: {k_val_0, k_val_1} = {32'h428a2f98, 32'h71374491}; // 64 hằng số K
            1: {k_val_0, k_val_1} = {32'hb5c0fbcf, 32'he9b5dba5};
            2: {k_val_0, k_val_1} = {32'h3956c25b, 32'h59f111f1};
            3: {k_val_0, k_val_1} = {32'h923f82a4, 32'hab1c5ed5};
            4: {k_val_0, k_val_1} = {32'hd807aa98, 32'h12835b01};
            5: {k_val_0, k_val_1} = {32'h243185be, 32'h550c7dc3};
            6: {k_val_0, k_val_1} = {32'h72be5d74, 32'h80deblfe};
            7: {k_val_0, k_val_1} = {32'h9bdc06a7, 32'hcl9bf174};
            8: {k_val_0, k_val_1} = {32'he49b69c1, 32'hefba4786};
            9: {k_val_0, k_val_1} = {32'h0fc19dc6, 32'h240calcc};
            10: {k_val_0, k_val_1} = {32'h2de92c6f, 32'h4a7484aa};
            11: {k_val_0, k_val_1} = {32'h5cb0a9dc, 32'h76f988da};
            12: {k_val_0, k_val_1} = {32'h983e5152, 32'ha831c66d};
            13: {k_val_0, k_val_1} = {32'hb00327c8, 32'hbf597fc7};
            14: {k_val_0, k_val_1} = {32'hc6e00bf3, 32'hd5a79147};
            15: {k_val_0, k_val_1} = {32'h06ca6351, 32'h14292967};
            16: {k_val_0, k_val_1} = {32'h27b70a85, 32'h2e1b2138};
            17: {k_val_0, k_val_1} = {32'h4d2c6dfe, 32'h53380d13};
            18: {k_val_0, k_val_1} = {32'h650a7354, 32'h766a0abb};
            19: {k_val_0, k_val_1} = {32'h81c2c92e, 32'h92722c85};
            20: {k_val_0, k_val_1} = {32'ha2bfe8a1, 32'ha81a664b};
            21: {k_val_0, k_val_1} = {32'hc24b8b70, 32'hc76c51a3};
            22: {k_val_0, k_val_1} = {32'hd192e819, 32'hd6990624};
            23: {k_val_0, k_val_1} = {32'hf40e3585, 32'h106aa070};
            24: {k_val_0, k_val_1} = {32'h19a4c116, 32'h1e376c08};
            25: {k_val_0, k_val_1} = {32'h2748774c, 32'h34b0bcb5};
            26: {k_val_0, k_val_1} = {32'h391c0cb3, 32'h4ed8aa4a};
            27: {k_val_0, k_val_1} = {32'h5b9cca4f, 32'h682e6ff3};
            28: {k_val_0, k_val_1} = {32'h748f82ee, 32'h78a5636f};
            29: {k_val_0, k_val_1} = {32'h84c87814, 32'h8cc70208};
            30: {k_val_0, k_val_1} = {32'h90bfeffa, 32'ha4506ceb};
            31: {k_val_0, k_val_1} = {32'hbef9a3f7, 32'hc67178f2};
            default: {k_val_0, k_val_1} = 64'b0;
        endcase
    end
```

Hình 4.1: Khối tạo hằng số K

Bộ đếm count [4:0]: Đây là tín hiệu đầu vào đóng vai trò tương tự như một địa chỉ (address) dùng để truy cập một cặp hằng số trong bộ nhớ ROM. Đầu vào này có độ rộng 5 bit ($2^5 = 32$ trạng thái), tương ứng với 32 chu kỳ xung nhịp phục vụ cho 64 vòng lặp

nén. Kỹ thuật này cho phép hệ thống truy xuất và cung cấp cùng lúc 2 hằng số trong 1 chu kỳ.

k_val_0 [31:0]: Là đầu ra hằng số K_t tương ứng với vòng chẵn (Ví dụ: K₀, K₂, K₄,...)

k_val_1 [31:0]: Là đầu ra hằng số K_{t+1} tương ứng với vòng lẻ (Ví dụ: K₁, K₃, K₅,...)

4.2 Khối Message

4.2.1 Khai báo Input/Output và thanh ghi

```
module message_expand(  
    input          clk,  
    input          reset,  
    input          data_en,  
    input [31:0]    block_in,  
    output reg      message_valid,  
    output [31:0]    debug_w,  
    output reg [31:0] w_data_0,  
    output reg [31:0] w_data_1  
);  
    reg [31:0] w[15:0]; // 16 thanh ghi đếm vòng  
    reg [5:0]  word_cnt; // biến đếm word  
    reg [3:0]  load_idx; // biến đếm 16 word đầu vào đầu tiên
```

Hình 4.2.1.1: Khai báo I/O và thanh ghi cho khối Message

Các input và chức năng:

- clk: Xung clock đồng bộ hóa hoạt động.
- reset: Tín hiệu khởi động lại mạch
- data_en: Tín hiệu cho phép nạp đầu vào và chuyển trạng thái (LOAD)
- block_in [31:0]: Dữ liệu đầu vào 32 bit.

Các output và chức năng:

message_valid: báo hiệu dữ liệu nạp xong.

w_data_0, w_data_1: xuất hai word liên tiếp trong quá trình mở rộng.

Thanh ghi:

w [15:0]: Đây là mảng thanh ghi lưu trữ 16 word đầu tiên, sau đó khi các word được mở rộng vẫn sẽ dùng các thanh ghi này để lưu

trữ các word tiếp theo theo cơ chế bộ đệm vòng.

word_cnt: Biến đếm phục vụ cho các vòng lặp tính toán và xả dữ liệu

load_idx: Con trỏ địa chỉ dùng trong quá trình nạp 16 từ đầu vào đầu tiên.

4.2.2 Các phép toán

Sử dụng các hàm xoay và dịch bit (σ_0 , σ_1). Trong thiết kế này, các hàm này được mô tả dưới dạng function:

```
// các hàm tính toán
function [31:0] rotr (input [31:0] x, input integer n);
    rotr = (x >> n) | (x << (32 - n));
endfunction

function [31:0] shr (input [31:0] x, input integer n);
    shr = (x >> n);
endfunction

function [31:0] sig0(input [31:0] x);
    sig0 = rotr(x, 7) ^ rotr(x, 18) ^ shr(x, 3);
endfunction

function [31:0] sig1(input [31:0] x);
    sig1 = rotr(x, 17) ^ rotr(x, 19) ^ shr(x, 10);
endfunction
```

Hình 4.2.2: Các phép toán trong khối Message

Hàm rotr giúp dịch x sang phải n vị trí ($x \gg n$) và phần bit được đẩy ra sẽ bị loại bỏ và dịch bit x sang trái ($32 - n$) vị trí. Đây chính là phần bit đã bị loại bỏ ở bước dịch x sang phải và sẽ được đưa vào lại phía bên trái.

Hàm shr là 1 phép dịch logic thông thường tương tự so với phép dịch vòng nhưng thay vì các bit bị dịch mất được xoay lại thì các bit bị dịch sẽ được thay bằng các bit 0.

Hàm rotr và shr được thiết kế bằng các toán tử dịch bit nhị phân ($\gg$, $\ll$) và cổng OR logic ($\mid$).

Các hàm sig0 và sig1 gọi lại rotr và shr kết hợp với cổng XOR ($\wedge$).

4.2.3 Kỹ thuật ánh xạ địa chỉ tuần hoàn

Để tính toán từ W_t , thuật toán chuẩn sử dụng công thức:

$$W_t = \sigma_1(W_{t-2}) + W_{t-7} + \sigma_0(W_{t-15}) + W_{t-16}$$

Tuy nhiên, do bộ đệm của chúng ta chỉ có 16 phần tử, việc gọi các chỉ số như $t-15$ hay $t-7$ sẽ gây lỗi tràn mảng bộ nhớ. Đoạn code của bạn đã giải quyết triệt để bài toán này bằng phép toán modulo địa chỉ:

```
wire [3:0] idx_w = word_cnt[3:0];
wire [31:0] new_w_0, new_w_1;

assign new_w_0 = sig1(w[idx_w + 4'd14]) +
                w[idx_w + 4'd9] +
                sig0(w[idx_w + 4'd1]) +
                w[idx_w];

assign new_w_1 = sig1(w[idx_w + 4'd15]) +
                w[idx_w + 4'd10] +
                sig0(w[idx_w + 4'd2]) +
                w[idx_w + 4'd1];
```

Khai báo `wire [3:0] idx_w = word_cnt[3:0];` giúp hệ thống tự động ép chỉ số về không gian 4-bit (tương đương với phép modulo 16). Biến `idx_w` lúc này đại diện cho vị trí cũ nhất trong bộ đệm, tức là tương đương với giá trị W_{t-16} .

Dựa trên mốc `idx_w` này, các khoảng cách trong công thức gốc được tịnh tiến lên tương ứng:

W_{t-16} ánh xạ thành `w[idx_w]`

W_{t-15} ánh xạ thành `w[(idx_w + 1) % 16]`

W_{t-7} ánh xạ thành `w[(idx_w + 9) % 16]`

W_{t-2} ánh xạ thành `w[(idx_w + 14) % 16]`

Để hỗ trợ kiến trúc Two-Stage Unrolling của khối Compression, mạch tổ hợp được thiết kế nhân đôi để giải quyết đồng thời hai phương trình:

- Mạch tổ hợp 1 (`new_w_0`): Tính toán cho từ W_t
- Mạch tổ hợp 2 (`new_w_1`): Tính toán cho từ W_{t+1} bằng cách tịnh tiến toàn bộ các chỉ số của `new_w_0` lên 1 đơn vị.

Hai khối cộng này hoạt động độc lập và song song. Kết quả `new_w_0` và `new_w_1` được tính toán xong ngay trong nửa đầu của chu kỳ xung nhịp và sẵn sàng để ghi đè ngược lại vào thanh ghi tại sườn lên tiếp theo.

4.2.4 Khối điều khiển (Finite State Machine – FSM)

```

localparam IDLE   = 2'b00;
localparam LOAD   = 2'b01;
localparam EXPAND = 2'b10;
localparam OUT    = 2'b11;

reg [1:0] current_st, next_st;

always @(*) begin
    next_st = current_st;
    case(current_st)
        IDLE: if(data_en) next_st = LOAD;
        LOAD: if(load_idx == 15) next_st = EXPAND;
        EXPAND: if(word_cnt == 46) next_st = OUT;
        OUT: if(word_cnt == 62) next_st = IDLE;
        default: next_st = IDLE;
    endcase
end

always @(posedge clk or negedge reset) begin
    if(!reset) current_st <= IDLE;
    else current_st <= next_st;
end

integer i;
always @(posedge clk or negedge reset) begin
    if(!reset) begin
        for(i=0; i<16; i=i+1) w[i] <= 32'b0;
        load_idx <= 0;
        word_cnt <= 0;
        w_data_0 <= 0;
        w_data_1 <= 0;
        message_valid <= 0;
    end else begin
        case(current_st)
            IDLE: begin
                load_idx <= 0;
                word_cnt <= 0;
                message_valid <= 0;
            end
            LOAD: begin
                w[load_idx] <= block_in;
                load_idx <= load_idx + 1;
                if(load_idx == 15) message_valid <= 1'b1;
            end
            EXPAND: begin
                w_data_0 <= w[idx_w];
                w_data_1 <= w[idx_w + 4'd1];
                w[idx_w] <= new_w_0;
                w[idx_w + 4'd1] <= new_w_1;
                word_cnt <= word_cnt + 2;
                message_valid <= 1'b0;
            end
            OUT: begin
                w_data_0 <= w[idx_w];
                w_data_1 <= w[idx_w + 4'd1];
                word_cnt <= word_cnt + 2;
            end
        endcase
    end
end
endmodule

```

Hình 4.2.4: Các trạng thái của Message

Để đồng bộ tuyệt đối với Khối Hàm nén (Compression Unit), máy trạng thái (FSM) của khối Message được thiết kế phân rã làm 4 giai đoạn, quản lý vòng đời của luồng dữ liệu từ lúc nạp thô cho đến khi nội suy và xả toàn bộ 64 từ ($W_0 - W_{63}$).

Trạng thái IDLE:

Mục đích: Đưa hệ thống về trạng thái an toàn.

Hoạt động: Khi mạch vừa được cấp nguồn, reset hoặc vừa xử lý xong 1 đoạn 512 bit thì hệ thống sẽ nằm ở trạng thái này. Các con trỏ địa chỉ được kéo về mức 0 ($load_idx \leq 0$, $word_cnt \leq 0$) và cờ $message_valid \leq 0$. FSM bị treo ở đây và chờ tín hiệu $data_en = 1$ để có thể chuyển sang trạng thái khác.

Trạng thái LOAD:

Mục đích: Nạp 16 từ đầu vào ban đầu ($W_0 - W_{15}$).

Hoạt động: Khi $data_en$ kích hoạt, FSM chuyển sang LOAD. Trong đúng 16 chu kỳ xung nhịp tiếp theo, tại mỗi sườn lên của clock, dữ liệu từ cổng $block_in$ được chốt vào mảng bộ đệm $w[load_idx]$, sau đó con trỏ $load_idx$ tăng lên 1 đơn vị ($load_idx \leq load_idx + 1$).

Chuyển trạng thái: Ở chu kỳ thứ 16, khi $load_idx == 15$, từ dữ liệu cuối cùng W_{15} được nạp. Mạch ngay lập tức bật cờ $message_valid \leq 1$ để đánh thức khối Compression ở đầu ra, và FSM chuyển sang EXPAND.

Trạng thái EXPAND:

Mục đích: Vừa xuất dữ liệu cho khối Compression, vừa tính toán mở rộng ra các từ dữ liệu mới.

Hoạt động: Bộ đếm $word_cnt$ khởi tạo từ 0. Tại mỗi chu kỳ, ba tác vụ vật lý diễn ra đồng thời nhờ đặc tính gán không đồng bộ ($\leq$) của Verilog:

- Xuất dữ liệu: Mạch đọc $w[idx_w]$ và $w[(idx_w + 1)]$ (với idx_w là 4 bit cuối của $word_cnt$) đẩy ra cổng w_data_0 và w_data_1 . Ở chu kỳ đầu tiên ($idx_w = 0$), mạch xuất chính xác W_0 và W_1
- Ghi đề: Đồng thời, các khối tổ hợp tính toán xong hai từ tương lai là W_{16} và W_{17} (thông qua tín hiệu dây new_w_0 , new_w_1). Hai từ mới này được ghi đề trực tiếp vào vị trí của W_0 và W_1 trong bộ đệm thông qua lệnh gán $w[idx_w] \leq new_w_0$ và $w[idx_w + 1] \leq new_w_1$. Việc đọc ra (xuất) và ghi đề diễn ra trong cùng một nhịp clock mà không gây xung đột dữ liệu.
- Tăng biến đếm: Bước nhảy của thuật toán là 2, nên $word_cnt \leq word_cnt + 2$.

Chuyển trạng thái: Giai đoạn này kéo dài trong 24 chu kỳ xung nhịp (với $word_cnt$ chạy các giá trị chẵn từ 0, 2, 4,... đến 46).

- Khi $word_cnt = 46$, mạch xuất ra W_{46} , W_{47} và vừa tính toán xong cặp từ cuối cùng của thuật toán là W_{62} , W_{63} .

- Lúc này, toàn bộ 64 từ dữ liệu đã được sinh ra trọn vẹn. FSM chuyển sang OUT.

Trạng thái OUT:

Mục đích: Xuất 16 từ dữ liệu cuối cùng ($W_{48} - W_{63}$) đã được lưu sẵn trong bộ đệm mà không kích hoạt khối tính toán tổ hợp.

Hoạt động: Do đã có đủ 64 từ, logic FSM ngắt các lệnh ghi đè vào mảng thanh ghi w, chỉ duy trì việc đọc và xuất dữ liệu ra w_data_0, w_data_1 dựa trên chỉ số modulo idx_w.

Chuyển trạng thái: Bộ đếm word_cnt tiếp tục tăng thêm 2 ($\text{word_cnt} \leq \text{word_cnt} + 2$).

- Khi $\text{word_cnt} == 62$, hai từ cuối cùng là W_{62}, W_{63} được đẩy ra cổng xuất. Hệ thống đã hoàn thành cung cấp luồng thông điệp. FSM tự động vòng về trạng thái IDLE để chuẩn bị cho một khối 512-bit mới.

4.3 Khối Compression

4.3.1 Khai báo Input/Output và thanh ghi

```
module compression(  
    input          clk,  
    input          reset,  
    input          message_valid,  
    input [31:0]   w_data_0,  
    input [31:0]   w_data_1,  
    output [255:0] hash_result,  
    output reg     hash_done  
);  
    // Thanh ghi làm việc  
    reg [31:0] a, b, c, d, e, f, g, h;  
    // Thanh ghi tích lũy (IV)  
    reg [31:0] hash_0, hash_1, hash_2, hash_3, hash_4, hash_5, hash_6, hash_7;  
  
    reg [4:0] count;  
    wire [31:0] k_val_0, k_val_1;
```

Hình 4.3.1: Khai báo I/O và thanh ghi khối Compression

clk : Tín hiệu vận hành xung nhịp hệ thống.

reset: Tín hiệu khởi động lại

message_valid: Tín hiệu được lấy từ khối Message, khi tín hiệu này được bật, khối Compression thoát khỏi trạng thái chờ và bắt đầu chu trình nén dữ liệu.

w_data_0 [31:0] và w_data_1 [31:0]: Hai bus dữ liệu cung cấp đồng thời hai từ thông điệp W_t và W_{t+1} từ khối Message. Đây là đầu vào bắt buộc để hiện thực hóa kiến trúc gỡ vòng (Unrolling).

hash_result [255:0] và hash_done: Đầu ra của thuật toán. Tín hiệu done sẽ được bật khi quá trình băm đã hoàn tất và sẽ xuất ra kết quả 256 bit băm cuối cùng.

Thanh ghi làm việc (a, b, c, d, e, f, g, h): Gồm 8 thanh ghi 32-bit dùng để lưu trữ trạng thái biến đổi liên tục qua từng vòng băm.

Thanh ghi lưu trữ Hash trung gian (hash_0 đến hash_7): Lưu trữ kết quả tích lũy của các khối dữ liệu trước đó. Việc tách biệt hai nhóm này cho phép hệ thống nạp dữ liệu mới vào các thanh ghi làm việc trong khi vẫn giữ được giá trị gốc để cộng dồn ở cuối chu trình.

Bộ đếm count [4:0]: Một bộ đếm 5-bit kiểm soát 32 chu kỳ xử lý. Vì mỗi chu kỳ xử lý 2 vòng nén, bộ đếm này chỉ cần chạy đến 31 là hoàn thành đủ 64 vòng băm của thuật toán chuẩn.

4.3.2 Các phép toán

Để thực hiện các vòng lặp băm, khối compression sử dụng một hệ thống các hàm logic tổ hợp và phép toán số học được đóng gói trong các function. Mạch thiết kế triển khai 5 hàm cốt lõi để tạo ra sự xáo trộn bit:

```
function [31:0] ch(input [31:0] x, y, z);  
    ch = (x & y) ^ (~x & z);  
endfunction  
  
function [31:0] maj(input [31:0] x, y, z);  
    maj = (x & y) ^ (x & z) ^ (y & z);  
endfunction  
  
function [31:0] rotr(input [31:0] x, input [4:0] n);  
    rotr = (x >> n) | (x << (32 - n));  
endfunction  
  
function [31:0] sig0(input [31:0] x);  
    sig0 = rotr(x, 2) ^ rotr(x, 13) ^ rotr(x, 22);  
endfunction  
  
function [31:0] sig1(input [31:0] x);  
    sig1 = rotr(x, 6) ^ rotr(x, 11) ^ rotr(x, 25);  
endfunction
```

Hình 4.3.2: Các phép toán khối Compression

Phép xoay vòng phải (rotr): Được hiện thực hóa bằng cách kết hợp phép dịch bit và cổng OR: $(x \gg n) | (x \ll (32 - n))$. Phép toán này đảm bảo thông tin vị trí bit được luân chuyển liên tục mà không bị mất mát dữ liệu.

Hàm Sigma(Σ_0, Σ_1): Đây là các hàm xáo trộn bit chính của khối nén, sử dụng tổ hợp XOR của ba phép xoay vòng phải với các khoảng cách khác nhau (2, 13, 22 cho Σ_0 và 6, 11, 25 cho Σ_1). Việc chọn các hằng số xoay này giúp tối đa hóa hiệu ứng thác (avalanche effect) trong mạch.

Hàm Lựa chọn (ch - Choose): Sử dụng các cổng AND, XOR và NOT: $(x \& y) \wedge (\sim x \& z)$. Hàm này đóng vai trò như một bộ ghép kênh mức bit (bit-level multiplexer), chọn bit từ y hoặc z dựa trên giá trị của x.

Hàm Đa số (maj - Majority): $(x \& y) \wedge (x \& z) \wedge (y \& z)$. Hàm này trả về giá trị xuất hiện nhiều nhất trong ba bit đầu vào, tạo ra tính phi tuyến mạnh mẽ cho quá trình nén.

4.3.3 Luồng tính toán 2 tầng (Two-Stage Computational Path)

Để thể hiện rõ cấu trúc gỡ vòng thông qua việc phân chia các biến tạm t1, t2 thành hai giai đoạn nối tiếp:

```
// round t
wire [31:0] sl_e = sigl(e);
wire [31:0] ch_efg = ch(e, f, g);
wire [31:0] kw0 = k_val_0 + w_data_0;
wire [31:0] t1 = (h + sl_e) + (ch_efg + kw0);

wire [31:0] s0_a = sig0(a);
wire [31:0] maj_abc = maj(a, b, c);
wire [31:0] t2 = s0_a + maj_abc;

wire [31:0] e_mid = d + t1;
wire [31:0] a_mid = t1 + t2;

// round t+1
wire [31:0] sl_e_mid = sigl(e_mid);
wire [31:0] ch_mid = ch(e_mid, e, f);
wire [31:0] kw1 = k_val_1 + w_data_1;

wire [31:0] t1_next = (g + sl_e_mid) + (ch_mid + kw1);

wire [31:0] s0_a_mid = sig0(a_mid);
wire [31:0] maj_mid = maj(a_mid, a, b);
wire [31:0] t2_next = s0_a_mid + maj_mid;

wire [31:0] a_next = t1_next + t2_next;
wire [31:0] e_next = c + t1_next;

assign hash_result = {hash_0, hash_1, hash_2, hash_3, hash_4, hash_5, hash_6, hash_7};
```

Hình 4.3.3: Tính toán 2 tầng

Tầng 1: Tính toán cho Vòng t

Tại đây, mạch tổ hợp tính toán dựa trên trạng thái hiện tại của các thanh ghi từ a đến h cùng cặp dữ liệu đầu tiên kw0 (w_data_0, k_val_0). Để giải quyết độ trễ lan truyền, cấu trúc Cây cộng song song (Adder Tree) được áp dụng triệt để:

- Biến tạm t1: Là kết quả tổng hợp của 5 toán hạng. Thay vì thực hiện chuỗi phép cộng nối tiếp, kiến trúc Cây cộng gom nhóm các toán hạng thành các nhánh tính toán đồng thời: $(h + \Sigma_1(e)) + (Ch(e, f, g) + kw_0)$. Nhờ vậy, mạch giảm thiểu đáng kể thời gian trễ trên đường găng (critical path).

- Biến tạm $t2$: Được tính toán độc lập và song song ở một nhánh khác từ $\Sigma0(a)$ và $maj(a, b, c)$.
- Kết quả trung gian: Hai bộ cộng cuối cùng của Tầng 1 sẽ tạo ra $e_mid = d + t1$ và $a_mid = t1 + t2$. Hai giá trị này đại diện cho trạng thái của thanh ghi e và a nếu hệ thống chốt dữ liệu sau một vòng đơn.

Tầng 2: Tính toán cho Vòng $t+1$

Thay vì chốt dữ liệu qua flip-flop, các tín hiệu điện tiếp tục đâm thẳng vào mạng logic của tầng thứ hai. Điểm mấu chốt là tầng này hấp thụ ngay lập tức các giá trị vừa sinh ra từ Tầng 1 để làm đầu vào:

- Các khối hàm tổ hợp ch và $sig1$ lúc này nhận tín hiệu chuyển tiếp là e_mid .
- Các khối hàm tổ hợp maj và $sig0$ nhận tín hiệu chuyển tiếp là a_mid .
- Biến $t1_next$ và $t2_next$: Tương tự như vòng trước, biến $t1_next$ đi qua một Cây cộng (Adder Tree) thứ hai của Tầng 2, tiêu thụ cặp dữ liệu $kw1$ tiếp theo (w_data_1, k_val_1) cùng trạng thái dịch chuyển logic của các thanh ghi còn lại (g, f, a, b).
- Kết quả cuối cùng: Tổng hợp thành $a_next = t1_next + t2_next$ và $e_next = c + t1_next$. Các giá trị này chính thức được nạp vào thanh ghi tại sườn lên tiếp theo của xung nhịp (clock).

Đặc tính cấu trúc và đồng bộ đa khối:

- Cộng Modulo 2^{32} : Toàn bộ các nút cộng (toán tử $+$) trong mạng lưới Adder Tree đều mặc định thực hiện theo Modulo 2^{32} do giới hạn độ rộng dây dẫn 32-bit. Đặc tính này giúp loại bỏ nhu cầu thiết kế các mạch kiểm tra cờ nhớ tràn (carry-out) phức tạp, từ đó tối ưu hóa tối đa diện tích cổng logic (Area) cho toàn bộ Cấu trúc Cây cộng.
- Cập nhật Hash cuối khối: Tại trạng thái UPDATE, mạch thực hiện cộng tích lũy giá trị của 8 thanh ghi vào các biến lưu trữ gốc $hash_0$ đến $hash_7$. Đây là bước chốt chặn quan trọng nhất để xâu chuỗi kết quả băm trung gian, đảm bảo tính liên tục của thuật toán trong trường hợp phải xử lý luồng thông điệp kéo dài qua nhiều khối (đa khối).

4.3.4 Khối điều khiển (Finite State Machine – FSM)

FSM của khối Compression điều khiển luồng nén qua 3 trạng thái chính:

```
assign hash_result = {hash_0, hash_1, hash_2, hash_3, hash_4, hash_5, hash_6, hash_7};

// IV Constants
localparam INIT_H0 = 32'h6a09e667, INIT_H1 = 32'hbb67ae85, INIT_H2 = 32'h3c6ef372, INIT_H3 = 32'ha54ff53a;
localparam INIT_H4 = 32'h510e527f, INIT_H5 = 32'h9b05688c, INIT_H6 = 32'h1f83d9ab, INIT_H7 = 32'h5be0cd19;

// FSM States
localparam IDLE = 2'b00;
localparam HASHING = 2'b10;
localparam UPDATE = 2'b11;

reg [1:0] current_st, next_st;

always @(*) begin
    next_st = current_st;
    case(current_st)
        IDLE: if(message_valid) next_st = HASHING;
        HASHING: if(count == 31) next_st = UPDATE;
        UPDATE: next_st = IDLE;
        default: next_st = IDLE;
    endcase
end
```

```

always @(posedge clk or negedge reset) begin
    if(!reset) begin
        {a, b, c, d, e, f, g, h} <= 256'b0;
        hash_done <= 1'b0;
        hash_0 <= INIT_H0;
        hash_1 <= INIT_H1;
        hash_2 <= INIT_H2;
        hash_3 <= INIT_H3;
        hash_4 <= INIT_H4;
        hash_5 <= INIT_H5;
        hash_6 <= INIT_H6;
        hash_7 <= INIT_H7;
        count <= 0;
    end else case(current_st)
        IDLE: begin
            hash_done <= 1'b0;
            count <= 0;
            if(message_valid) begin
                a <= hash_0;
                b <= hash_1;
                c <= hash_2;
                d <= hash_3;
                e <= hash_4;
                f <= hash_5;
                g <= hash_6;
                h <= hash_7;
            end
        end
    endcase

    HASHING: begin
        // Dịch thanh ghi nhảy 2 bậc (Jump Shifting)
        h <= f;
        g <= e;
        f <= e_mid;
        e <= e_next;
        d <= b;
        c <= a;
        b <= a_mid;
        a <= a_next;
        count <= count + 1;
    end

    UPDATE: begin
        hash_done <= 1'b1;
        // Cong don Modulo 2^32
        hash_0 <= a + hash_0;
        hash_1 <= b + hash_1;
        hash_2 <= c + hash_2;
        hash_3 <= d + hash_3;
        hash_4 <= e + hash_4;
        hash_5 <= f + hash_5;
        hash_6 <= g + hash_6;
        hash_7 <= h + hash_7;
    end
endcase
end
endmodule

```

Hình 4.3.4: Các trạng thái của Compression

Khi hệ thống vừa cấp nguồn hoặc tín hiệu reset bị kéo xuống mức 0, toàn bộ mạch phần cứng sẽ được làm sạch:

- Các thanh ghi trạng thái làm việc a đến h và bộ đếm count bị xóa về 0 .
- Tín hiệu báo hiệu hoàn tất done bị ngắt (hash_done <= 0).
- Đặc biệt, hệ thống tự động nạp các hằng số tiêu chuẩn của thuật toán SHA-256 (được định nghĩa bởi NIST FIPS 180-4) vào các thanh ghi lưu trữ Hash trung gian hash_0 - hash_7. Các hằng số này (ví dụ: INIT_H0 = 32'h6a09e667) đóng vai trò là Véc-tơ khởi tạo (IV) cho khối dữ liệu đầu tiên

Phân tích chi tiết các trạng thái:

Trạng thái IDLE:

Hoạt động: Tại IDLE, hệ thống duy trì hash_done <= 0 và bộ đếm count <= 0 . FSM liên tục lấy tín hiệu message_valid từ khối Message.

Điều kiện chuyển trạng thái: Chỉ khi valid_out == 1, FSM mới quyết định chuyển sang trạng thái HASHING ở chu kỳ tiếp theo .

Thao tác nạp song song: Ngay tại thời điểm nhận cờ message_valid == 1, mạch thực hiện nạp đồng loạt 8 giá trị từ mảng hash_0...hash_7 vào 8 thanh ghi làm việc a - h .

Thao tác này chỉ tốn đúng 1 chu kỳ xung nhịp, chuẩn bị sẵn sàng toán hạng cho vòng nén đầu tiên.

Trạng thái HASHING:

Hoạt động: Đây là trạng thái tiêu tốn thời gian nhất của khối nén. Tại mỗi nhịp xung đồng hồ, bộ đếm count tăng dần lên 1 đơn vị ($\text{count} \leq \text{count} + 1$). Đồng thời, các thanh ghi a - h liên tục cập nhật trạng thái mới theo cơ chế "dịch nhảy 2 bậc" ($h \leq f$, $g \leq e$, $f \leq e_{\text{mid}}$, $e \leq e_{\text{next}}$...). Quá trình này diễn ra hoàn toàn tự động, hút dữ liệu từ mạng lưới Cây cộng tổ hợp (Adder Tree) ở các biểu thức a_{next} , e_{next} nạp vào flip-flop.

Điều kiện chuyển trạng thái: FSM liên tục kiểm tra giá trị của bộ đếm. Nếu $\text{count} < 31$, FSM giữ nguyên ở HASHING. Khi $\text{count} == 31$, tức là hệ thống đã thực hiện xong 32 chu kỳ vật lý (tương đương 64 vòng bấm logic nhờ Unrolling), mạch tổ hợp Next State Logic sẽ ép FSM chuyển thẳng sang trạng thái UPDATE.

Trạng thái UPDATE:

Hoạt động: Trạng thái này có nhiệm vụ kết thúc chu trình của một khối 512-bit.

Tín hiệu done được kéo lên mức 1 ($\text{hash_done} \leq 1$) để báo hiệu cho vi xử lý bên ngoài hoặc Top-level biết chuỗi bấm 256-bit đã sẵn sàng.

Đồng thời, mạch thực hiện chuỗi 8 phép cộng đồng thời: lấy trạng thái cuối cùng của a - h cộng dồn (modulo 2^{32}) vào hash_0 - hash_7 . Thao tác này chốt lại bản bấm trung gian.

Điều kiện chuyển trạng thái: FSM không có lệnh rẽ nhánh điều kiện ở trạng thái này. Nó chỉ dừng lại ở OUTPUT đúng 1 chu kỳ clock, sau đó bị ép quay trở lại trạng thái IDLE một cách vô điều kiện ($\text{next_st} = \text{IDLE}$). Việc tự động vòng về IDLE ngay lập tức giúp hệ thống sẵn sàng "đón" cờ message_valid của khối thông điệp thứ hai mà không bị lãng phí bất kỳ chu kỳ xung nhịp nào.

4.4 SHA_Top Module

Đây là khối đóng vai trò là khối top – level trong sơ đồ phân cấp RTL. Ở khối này, hệ thống không chứa các khối logic tổ hợp hay thanh ghi mà sử dụng phương pháp mô tả cấu trúc (Structural Modeling). Nhiệm vụ của nó là khởi tạo 2 module lõi là Message và Compression và định tuyến các đường tín hiệu giữa chúng để tạo thành 1 khối gia tốc độc lập hoàn chỉnh (SHA – 256).

```

module top_sha256 (
    input wire          clk,
    input wire          reset,
    input wire          data_en,
    input wire [31:0]   block_in,
    output wire [31:0]  debug_w,
    output wire [255:0] hash_result,
    output wire          hash_done
);
    wire          valid_wire;
    wire [31:0]   w_bus_0;
    wire [31:0]   w_bus_1;
    message_expand u_message (
        .clk(clk),
        .reset(reset),
        .data_en(data_en),
        .block_in(block_in),
        .message_valid(valid_wire),
        .debug_w(debug_w),
        .w_data_0(w_bus_0),
        .w_data_1(w_bus_1)
    );
    compression u_compression (
        .clk(clk),
        .reset(reset),
        .message_valid(valid_wire),
        .w_data_0(w_bus_0),
        .w_data_1(w_bus_1),
        .hash_result(hash_result),
        .hash_done(hash_done)
    );
endmodule

```

Hình 4.4: Khối SHA_Top Module

Nhóm tín hiệu	Tên cổng	Độ rộng	Chiều (I/O)
Đồng bộ	clk	1 bit	Input
Bất đồng bộ	reset	1 bit	Input
Điều khiển	data_en	1 bit	Input
Dữ liệu vào	block_in	32 bit	Input
Trạng thái	hash_done	1 bit	Output
Dữ liệu ra	hash_result	256 bit	Output
Gỡ lỗi	debug_w	32 bit	Output

Bảng 4.4.1: Các chân tín hiệu giao tiếp của khối top_sha256

Để hiện thực hóa kiến trúc Two-Stage Unrolling ở cấp độ hệ thống, Top-Module sử dụng các tín hiệu dây dẫn (wire) bằng thông rộng để liên kết hai máy trạng thái (FSM) đang chạy song song

Tín hiệu nội bộ	Module Phát	Module Thu	Vai trò
valid_wire	u_message	u_compression	Cờ "Bắt tay" (Handshake) cấp hệ thống. Đảm bảo Hàm nén

			chỉ thức dậy khi khối Thông điệp đã nạp đủ 16 từ ban đầu.
w_bus_0	u_message	u_compression	Đường truyền tải trực tiếp từ thông điệp W_t (32-bit).
w_bus_1	u_message	u_compression	Đường truyền tải trực tiếp từ thông điệp W_{t+1} (32-bit).

Bảng 4.4.2: Tín hiệu định tuyến nội bộ của khối sha_top

Chương V. MÔ PHỎNG VÀ KẾT QUẢ

5.1 Thiết lập môi trường và phương pháp kiểm chứng

Để đánh giá tính đúng đắn của toàn bộ vi kiến trúc SHA-256 (bao gồm khối Message Expander, khối Compression và FSM điều khiển), hệ thống được mô phỏng trên công cụ ModelSim. File testbench tb_top_sha256.v được xây dựng để cung cấp môi trường kiểm chứng tự động.

5.2 Mô phỏng Pre-Simulation

```
`timescale 1ns / 1ps

module tb_top_sha256;
    localparam CLK_PERIOD = 20;

    reg          clk;
    reg          reset;
    reg          data_en;
    reg [31:0]    block_in;
    wire         hash_done;
    wire [255:0] hash_result;
    wire [31:0]   debug_w;

    reg [511:0] test_block_512;
    reg [255:0] expected_hash;

    integer test_count;
    integer pass_count;

    top_sha256 dut (
        .clk(clk),
        .reset(reset),
        .data_en(data_en),
        .block_in(block_in),
        .debug_w(debug_w),
        .hash_result(hash_result),
        .hash_done(hash_done)
    );
```

```

initial begin
    clk = 0;
    forever #(CLK_PERIOD/2) clk = ~clk;
end

task sys_reset;
begin
    reset = 1'b0;
    data_en = 1'b0;
    block_in = 32'b0;
    #(CLK_PERIOD * 2);
    reset = 1'b1;
    #(CLK_PERIOD);
end
endtask

task inject_512b_block;
input [511:0] block;
input [255:0] expected;
input is_final_block;
input [1023:0] test_name;

integer i;
begin
    @(negedge clk);
    data_en = 1'b1;
    @(negedge clk);

    // Nạp 16 tu 32 bit
    for (i = 0; i < 16; i = i + 1) begin
        block_in = block[511 - (i * 32) -: 32];
    end
end
endtask

@(posedge clk);
data_en = 1'b0;
block_in = 32'b0;

// Cho co hoan tat
while (!hash_done) begin
    @(posedge clk);
end

// So sanh ket qua
if (is_final_block) begin
    test_count = test_count + 1;
    if (hash_result === expected) begin
        pass_count = pass_count + 1;
        $display("Test %0d Thanh Cong: %0s", test_count, test_name);
    end else begin
        $display("Test %0d That Bai: %0s", test_count, test_name);
        $display("Mong doi: %h", expected);
        $display("Thuc te: %h", hash_result);
    end
end
end
@(posedge clk);
end
endtask

```

Hình 5.1.1: Testbench thực hiện mô phỏng Pre-Simulation

Xung nhịp (Clock): Chu kỳ xung nhịp được thiết lập ở mức CLK_PERIOD = 20 (đơn vị ns) , tương đương với tần số hoạt động 50MHz.

Tín hiệu Reset: Tuân thủ chuẩn thiết kế an toàn, hệ thống sử dụng Reset bất đồng bộ tích cực mức thấp (active-low). Trong testbench, quy trình này được mô phỏng qua task sys_reset, trong đó reset bị kéo xuống 1'b0 trong 2 chu kỳ clock để xả trạng thái, sau đó kéo lên 1'b1 để mạch bắt đầu hoạt động .

Task nạp dữ liệu (inject_512b_block): Để mô phỏng chân thực luồng dữ liệu truyền vào hệ thống, task này thực hiện vòng lặp 16 lần để nạp tuần tự 16 từ 32-bit (tạo thành một khối 512-bit) vào cổng block_in tại mỗi sườn lên của xung nhịp clock . Sau khi nạp xong, hệ thống sẽ ngắt tín hiệu data_en và chuyển sang chế độ chờ cờ hash_done từ module Top .

Cơ chế tự kiểm tra (Self-Checking): Testbench tích hợp biến test_count và pass_count để tự động theo dõi tiến độ . Khi cờ hash_done bật sáng, hệ thống so sánh trực tiếp kết quả băm hash_result với mã expected (mã băm này được sử dụng trên trang web mô

phòng thứ ba). Nếu trùng khớp, hệ thống in ra thông báo PASSED lên màn hình Console; ngược lại, in ra FAILED cùng với giá trị thực tế.

5.2.1 Kịch bản mô phỏng

Để chứng minh tính đúng của mạch, kịch bản mô phỏng được chia thành 4 nhóm chính:

Nhóm 1: Các trường hợp đơn giản và biên

Chuỗi rỗng (Empty String): Kiểm tra khả năng xử lý của mạch khi không có dữ liệu thực, chỉ có bit 1 đệm (0x80) và chiều dài chuỗi bằng 0.

Ký tự đơn: Chuỗi "a" để kiểm tra tính chính xác cơ bản.

Tràn block cứng: Chuỗi 55 ký tự "A" (vừa khít giới hạn padding của 1 block) và chuỗi 56 ký tự "B" (buộc mạch phải tự động ngắt và chuyển phần đệm sang block thứ 2).

```
// chuỗi rỗng ""
sys_reset();
expected_hash = 256'he3b0c44298f0c1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855;
test_block_512 = {32'h80000000, {14{32'h0}}, 32'h00000000};
inject_512b_block(test_block_512, expected_hash, 1'b1, "Empty String");

// a
sys_reset();
expected_hash = 256'hca978112calbbdcfac231b39a23dc4da786eff8147c4e72b9807785afee48bb;
test_block_512 = {32'h61800000, {14{32'h0}}, 32'h00000008};
inject_512b_block(test_block_512, expected_hash, 1'b1, "a");

// 55 chu A
sys_reset();
expected_hash = 256'h8963ccc0afd622cc7574ac2011f93a3059b3d65548a77542a1559e3d202e6ab00;
test_block_512 = {{13{32'h41414141}}, 32'h41414180, 32'h00000000, 32'h000001b8};
inject_512b_block(test_block_512, expected_hash, 1'b1, "55 chu A");

//56 chu B
sys_reset();
expected_hash = 256'h821c30ffb748ac6d776ad4972a6cbc7ca32e6aaf63b68808e7fe92321dfbb6b8; |
test_block_512 = {{14{32'h42424242}}, 32'h80000000, 32'h00000000};
inject_512b_block(test_block_512, expected_hash, 1'b0, "56 chu B");
test_block_512 = {{15{32'h0}}, 32'h000001c0};
inject_512b_block(test_block_512, expected_hash, 1'b1, "56 chu B");
```

Hình 5.2.1.1: Các kịch bản ví dụ mô phỏng Pre-Simulation cho trường hợp đơn giản và biên

Nhóm 2: Các trường hợp có đầu vào là 1 Block

Các chuỗi văn bản đơn giản có độ dài không quá 1 khối dữ liệu 512 bit


```
// Khoi
sys_reset();
expected_hash = 256'hf503c345fac9f94a562302625e7082390b7796b57f5e1559ab2dabfa4a3bfl3d;
test_block_512 = {32'h4b686f69, 32'h80000000, {13{32'h0}}, 32'h00000020};
inject_512b_block(test_block_512, expected_hash, 1'b1, "Khoi");

// Do an 1
sys_reset();
expected_hash = 256'h1763bdc10a961a4fed6b13a39212cdc2e35031ec66715a084a470e0f5531d784;
test_block_512 = {32'h446f2061, 32'h6e203180, {13{32'h0}}, 32'h00000038};
inject_512b_block(test_block_512, expected_hash, 1'b1, "Do an 1");

// IC Design"
sys_reset();
expected_hash = 256'h1690091835cc268e15fa4d05d6545f83a749814373e582fda067c388fle9e129;
test_block_512 = {32'h49432044, 32'h65736967, 32'h6e800000, {12{32'h0}}, 32'h00000048};
inject_512b_block(test_block_512, expected_hash, 1'b1, "IC_Design");

// FPGA
sys_reset();
expected_hash = 256'hdcbfca4bcd0a7f278db6959ad82411b2d20a9ea31e25f1d389e41ce23a89050;
test_block_512 = {32'h46504741, 32'h80000000, {13{32'h0}}, 32'h00000020};
inject_512b_block(test_block_512, expected_hash, 1'b1, "FPGA");

// Verilog
sys_reset();
expected_hash = 256'h25d9678d5751122ddd45aa87a465cab682e00ffa6a6ddf821fbd7a43fe8d200c;
test_block_512 = {32'h56657269, 32'h6c6f6780, {13{32'h0}}, 32'h00000038};
inject_512b_block(test_block_512, expected_hash, 1'b1, "Verilog");
```

Hình 5.2.1.2: Các kịch bản ví dụ mô phỏng Pre-Simulation cho trường hợp 1 block

Nhóm 3: Chuỗi tiếng Việt có dấu

Kiểm tra khả năng tương thích của mạch với bộ mã hóa UTF-8 (khiến mỗi ký tự có dấu chiếm nhiều byte hơn).

```
// Đại học CNTT
sys_reset();
expected_hash = 256'h2346dd071e2edfd3277da5415d76d59612219932254957f3e21ea5a7b281aca2;
test_block_512 = {32'hc490e1ba, 32'ha1692068, 32'helbb8d63, 32'h20434e54, 32'h54800000, {10{32'h0}}, 32'h00000088};
inject_512b_block(test_block_512, expected_hash, 1'b1, "??i h?c CNTT ");
// Sinh viên Lê Đăng Khôi
sys_reset();
expected_hash = 256'h96a526cd716626d303958e1e2d668ab9e2036f044f154dc97738227a9fd6alc8;
test_block_512 = {32'h53696e68, 32'h207669c3, 32'haa6e204c, 32'hc3aa20c4, 32'h90c4836e, 32'h67204b68, 32'hc3b46980, {8{32'h0}}, 32'h00000000};
inject_512b_block(test_block_512, expected_hash, 1'b1, "Sinh viên Lê ??ng Khôi");
// Sinh viên Võ Nguyễn Anh Khôi
sys_reset();
expected_hash = 256'h9ee035a7f54f12419a54c6b41a8635f93af54e7e635d46137489dc58b998f3e5;
test_block_512 = {32'h53696e68, 32'h207669c3, 32'haa6e2056, 32'hc3b5204e, 32'h677579e1, 32'hbb856e20, 32'h416e6820, 32'h4b68c3b4, 32'h00000000};
inject_512b_block(test_block_512, expected_hash, 1'b1, "Sinh viên Võ Nguy?n Anh Khôi");
```

Hình 5.2.1.3: Các kịch bản ví dụ mô phỏng Pre-Simulation cho trường hợp chuỗi tiếng Việt có dấu

Nhóm 4: Chuỗi các văn bản có chiều dài nhiều hơn 1 block 512 bit

Đây là nhóm kiểm thử quan trọng nhất để xác minh khả năng **cộng dồn Hash trung gian** của khối Compression khi phải xử lý các thông điệp dài bị cắt thành nhiều khối 512-bit liên tiếp.

Báo cáo Đồ án Thiết kế hệ thống số với HDL

```
// Viet Nam la mot quoc gia nam o ban dao Dong Duong, thuoc khu vuc Dong Nam A. Voi duong bo bien dai hon 3260 km, Viet Nam co t:
sys_reset();
expected_hash = 256'h2e42af125599c90d922a5d2bf9be09c892a5896035f1288a40af98e3630ef6f9;
test_block_512 = {32'h56696574, 32'h204e616d, 32'h206c6120, 32'h6d6f7420, 32'h71756f63, 32'h20676961, 32'h206e616d, 32'h206f2062,
inject_512b_block(test_block_512, expected_hash, 1'b0, "Viet Nam la mot quoc gia...");

test_block_512 = {32'h20446f6e, 32'h67204e61, 32'h6d20412e, 32'h20566f69, 32'h2064756f, 32'h6e672062, 32'h6f206269, 32'h656e2064,
inject_512b_block(test_block_512, expected_hash, 1'b0, "Viet Nam la mot quoc gia...");

test_block_512 = {32'h206e616e, 32'h67206c6f, 32'h6e207665, 32'h20706861, 32'h74207472, 32'h69656e20, 32'h6b696e68, 32'h20746520,
inject_512b_block(test_block_512, expected_hash, 1'b0, "Viet Nam la mot quoc gia...");

test_block_512 = {32'h204e616d, 32'h206c6120, 32'h4861204e, 32'h6f692c20, 32'h7468616e, 32'h68207068, 32'h6f206c6f, 32'h6e206e68,
inject_512b_block(test_block_512, expected_hash, 1'b1, "Viet Nam la mot quoc gia...");
//TroI hom nay trong xanh va co ve rat mat me. Toi ngoi trong quan ca phe quen thuoc tren con duong nho vang lang. Tieng nhac du
sys_reset();
expected_hash = 256'h54726f69, 32'h20686f6d, 32'h206e6179, 32'h2074726f, 32'h6e672078, 32'h616e6820, 32'h76612063, 32'h6f207665,
inject_512b_block(test_block_512, expected_hash, 1'b0, "TroI hom nay...");

test_block_512 = {32'h20636120, 32'h70686520, 32'h7175656e, 32'h20746875, 32'h6f632074, 32'h72656e20, 32'h636f6e20, 32'h64756f6e,
inject_512b_block(test_block_512, expected_hash, 1'b0, "TroI hom nay...");

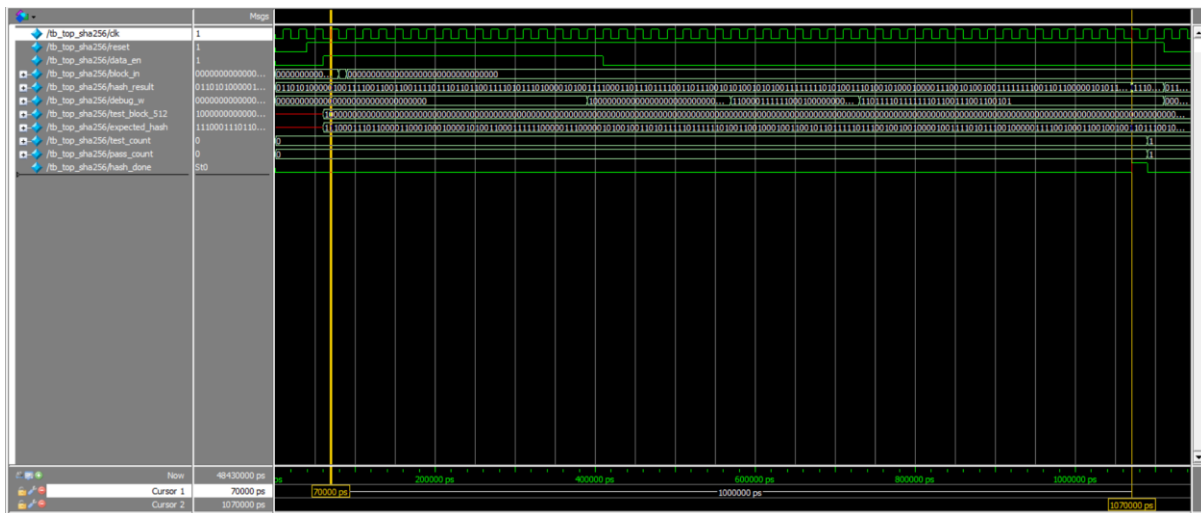
test_block_512 = {32'h756f6e67, 32'h2076616e, 32'h67206c65, 32'h6e207475, 32'h20636869, 32'h6563206c, 32'h6f612063, 32'h752c2068,
inject_512b_block(test_block_512, expected_hash, 1'b0, "TroI hom nay...");

test_block_512 = {32'h6f616920, 32'h63756120, 32'h736f2e20, 32'h4d6f7420, 32'h6e676179, 32'h2063756f, 32'h69207475, 32'h616e2074,
inject_512b_block(test_block_512, expected_hash, 1'b0, "TroI hom nay...");

test_block_512 = {32'h20686f6e, 32'h2074726f, 32'h206e656e, 32'h206e6865, 32'h206e6861, 32'h6e672068, 32'h6f6e2e80, 32'h00000000,
inject_512b_block(test_block_512, expected_hash, 1'b1, "TroI hom nay...");
```

Hình 5.2.1.4: Các kịch bản ví dụ mô phỏng Pre-Simulation cho trường hợp nhiều hơn 1 block

5.2.2 Kết quả mô phỏng Pre-Simulation



Hình 5.2.2.1: Kết quả mô phỏng dạng sóng Pre-Simulation

Dựa vào kết quả phân tích dạng sóng ta thấy được tín hiệu data_en được chốt vào hệ thống tại sườn lên của xung nhịp ở thời điểm 70000ps. Sau khi hoàn tất nạp dữ liệu và thực thi 32 vòng băm (được rút gọn bằng kiến trúc gỡ vòng 2 bước), tín hiệu hash_done được bật lên 1 tại thời điểm 1070000ps.

Vậy tổng thời gian thực thi logic là: $1070000\text{ps} - 70000\text{ps} = 1000000\text{ps}$. Với chu kỳ xung nhịp được thiết lập trong testbench là 20ns, tổng số chu kỳ hệ thống chạy là: $1000000\text{ps} / 20\text{ns} = 50$ (chu kỳ).

```

# ----- CHAY MO PHONG -----
# Test 1 Thanh Cong: Emty String
# Test 2 Thanh Cong: a
# Test 3 Thanh Cong: UIT
# Test 4 Thanh Cong: 55 chu A
# Test 5 Thanh Cong: 56 chu B
# Test 6 Thanh Cong: Khoi
# Test 7 Thanh Cong: Do an 1
# Test 8 Thanh Cong: IC_Design
# Test 9 Thanh Cong: FPGA
# Test 10 Thanh Cong: Verilog
# Test 11 Thanh Cong: Quartus
# Test 12 Thanh Cong: ModelSim
# Test 13 Thanh Cong: CE UIT
# Test 14 Thanh Cong: KIM
# Test 15 Thanh Cong: 4 chu A
# Test 16 Thanh Cong: 8 chu A
# Test 17 Thanh Cong: 12 chu A
# Test 18 Thanh Cong: Hanta virus
# Test 19 Thanh Cong: Cybersecurity Zero Trust
# Test 20 Thanh Cong: Nam quoc son ha Nam de cu. Tiet nien dinh phan tai thien thu.
# Test 21 Thanh Cong: Buoc toi Deo Ngang bong xe ta. Co cay chen da la chen hoa.
# Test 22 Thanh Cong: Do an duoc thuc hien duoi su huong dan cua giang vien Ngo Hieu Truong
# Test 23 Thanh Cong: Bao cao do an HDL vao ngay 29 thang 5
# Test 24 Thanh Cong: yáyýýáo ??ýán HDL
# Test 25 Thanh Cong: ??i h?c CNTT
# Test 26 Thanh Cong: Sinh viên yê ??ng yýôi
# Test 27 Thanh Cong: Sinhýýýýýýýýýý Nguy?n Anh ýýôi
# Test 28 Thanh Cong: Tech_Blockchain_blk4
# Test 29 Thanh Cong: Viet Nam la mot quoc gia...
# Test 30 Thanh Cong: Troi hom nay...

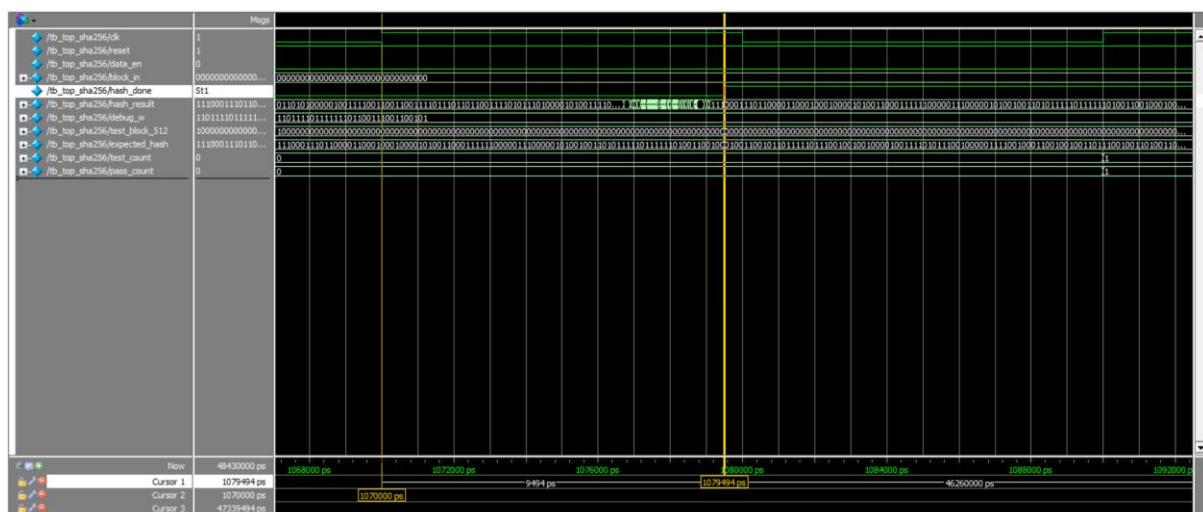
```

Hình 5.2.2.2: Kết quả mô phỏng Pre-Simulation hiển thị trên transcript

Qua quan sát trên dạng song và cửa sổ transcript cho thấy được hệ thống hoạt động đúng như mong đợi. Toàn bộ các Test Cases, từ chuỗi 0 byte đến các chuỗi đa khối kéo dài hàng trăm byte, đều cho ra kết quả hiển thị là “Thanh Cong”, chứng minh hệ thống không sai sót bất kỳ một bit nào so với thuật toán SHA-256 chuẩn của NIST.

Quá trình mô phỏng khẳng định lõi IP SHA-256 gia tốc phần cứng bằng kiến trúc Two-Stage Unrolling đã hoạt động hoàn toàn chính xác theo đúng đặc tả ban đầu, đáp ứng được khả năng tiếp nhận luồng dữ liệu liên tục và giải quyết tốt các trường hợp dị biệt của dữ liệu đầu vào.

5.3 Mô phỏng Post-Simulation



Hình 5.3.1: Kết quả mô phỏng dạng sóng Post-Simulation

Để đánh giá sát nhất với điều kiện hoạt động thực tế của mạch, hệ thống đã được tiến hành mô phỏng sau đi dây (Gate-Level Simulation) bằng cách nạp file trễ định tuyến chuẩn SDF (Standard Delay Format). Kết quả phân tích dạng sóng cho thấy những đặc tính vật lý rõ rệt của mạng lưới logic:

Đo lường độ trễ lan truyền (Propagation Delay) Thông qua công cụ đo lường thời gian (Cursor) trên ModelSim, khoảng thời gian trễ từ thời điểm xung nhịp clk được kích hoạt (sườn lên) cho đến khi ngõ ra báo hiệu hash_done cập nhật trạng thái đã được ghi nhận.

Kết quả đo: Khoảng cách giữa Cursor 1 (đặt tại sườn lên của clk lúc 1070000 ps) và Cursor 2 (đặt tại sườn lên của hash_done lúc 1079023 ps) cho ra giá trị sai lệch thời gian là Delta = 9494 ps (tương đương 9.494 ns).

Đánh giá: Khác với mô phỏng mức RTL (Pre-Simulation) mang tính lý tưởng với độ trễ bằng 0, con số 9.494 ns phản ánh chính xác tổng thời gian trễ vật lý (bao gồm Gate delay và Routing delay) khi tín hiệu phải truyền qua mạng lưới Cây cộng song song (Adder Tree) của kiến trúc Unrolling. Với chu kỳ xung nhịp hệ thống được thiết lập là 20 ns (tương đương tần số 50 MHz), mức trễ 9.494 ns nằm ở ngưỡng an toàn, đảm bảo mạch thỏa mãn các ràng buộc về thời gian (Setup time / Hold time) mà không gây ra bất kỳ vi phạm Timing nào.

```
# Test 1 Thanh Cong: Empty String
# Test 2 Thanh Cong: a
# Test 3 Thanh Cong: UIT
# Test 4 Thanh Cong: 55 chu A
# Test 5 Thanh Cong: 56 chu B
# Test 6 Thanh Cong: Khoi
# Test 7 Thanh Cong: Do an 1
# Test 8 Thanh Cong: IC_Design
# Test 9 Thanh Cong: FPGA
# Test 10 Thanh Cong: Verilog
# Test 11 Thanh Cong: Quartus
# Test 12 Thanh Cong: ModelSim
# Test 13 Thanh Cong: CE UIT
# Test 14 Thanh Cong: KIM
# Test 15 Thanh Cong: 4 chu A
# Test 16 Thanh Cong: 8 chu A
# Test 17 Thanh Cong: 12 chu A
# Test 18 Thanh Cong: Hanta virus
# Test 19 Thanh Cong: Cybersecurity Zero Trust
# Test 20 Thanh Cong: Nam quoc son ha Nam de cu. Tiet nhien dinh phan tai thien thu.
# Test 21 Thanh Cong: Buoc toi Deo Ngang bong xe ta. Co cay chen da la chen hoa.
# Test 22 Thanh Cong: Do an duoc thuc hien duoi su huong dan cua giang vien Ngo Hieu Truong
# Test 23 Thanh Cong: Bao cao do an HDL vao ngay 29 thang 5
# Test 24 Thanh Cong: YéYéYáo ??Yán HDL
# Test 25 Thanh Cong: ??i h?c CNTT
# Test 26 Thanh Cong: Sinh viên yê ??ng yýôi
# Test 27 Thanh Cong: SinhYéYéYéYéYé Nguy?n Anh yýôi
# Test 28 Thanh Cong: Tech_Blockchain_blk4
# Test 29 Thanh Cong: Viet Nam la mot quoc gia...
# Test 30 Thanh Cong: Troi hom nay...
# Break in Module tb_top_sha256 at C:/Users/Admin/Downloads/DoAnHDL/DoAnFinal/tb_top_sha256.v line 301
```


Hình 5.3.2: Kết quả mô phỏng Pre-Simulation hiển thị trên transcript

Bên cạnh việc phân tích dạng sóng trực quan, tính đúng đắn của hệ thống được kiểm chứng tự động và toàn diện thông qua cơ chế Self-Checking của Testbench, với kết quả được ghi nhận trực tiếp trên cửa sổ transcript của ModelSim.

Thông qua báo cáo hệ thống từ ModelSim, lõi IP SHA-256 được thiết kế đã chứng minh độ tin cậy tuyệt đối về mặt chức năng. Thuật toán phần cứng xuất ra kết quả tương đương 100% so với phần mềm tính toán tiêu chuẩn của NIST, sẵn sàng cho các bước kiểm thử trên thiết bị FPGA vật lý.

5.4 Tổng hợp phần cứng và đánh giá tài nguyên

Sau khi kiểm chứng tính đúng đắn của toàn bộ kiến trúc trên môi trường mô phỏng ModelSim, thiết kế mã nguồn RTL tiếp tục được đưa vào quy trình tổng hợp (Synthesis) và đi dây (Place & Route) thông qua công cụ phần mềm Quartus.

Kit được hiện thực cho đồ án này là kit ALTERA DE2, được vận hành trên chip Cyclone II.

Chi phí phần cứng và hiệu năng hoạt động của hệ thống SHA-256 sau khi tích hợp kỹ thuật gỡ vòng 2 bước, các thông số được đo lường và trích xuất từ báo cáo Compilation Report và Timing Analyzer của Quartus:

	Resource	Usage
1	Estimated Total logic elements	3,437
2		
3	Total combinational functions	2989
4	Logic element usage by number of LUT inputs	
1	-- 4 input functions	1114
2	-- 3 input functions	1775
3	-- <=2 input functions	100
5		
6	Logic elements by mode	
1	-- normal mode	2113
2	-- arithmetic mode	876
7		
8	Total registers	1111
1	-- Dedicated logic registers	1111
2	-- I/O registers	0
9		
10	I/O pins	324
11	Embedded Multiplier 9-bit elements	0
12	Maximum fan-out node	clk
13	Maximum fan-out	1111
14	Total fan-out	14795
15	Average fan-out	3.34

	Fmax	Restricted Fmax	Clock Name	Note
1	54.59 MHz	54.59 MHz	clk	

Hình 5.4.1: Báo cáo tài nguyên sử dụng trên kit ALTERA DE2

Tài nguyên	Thông số thực tế	Ghi chú
Tổng phần tử logic (Estimated ALUTs)	3,437	Tài nguyên tổng thể mà lõi IP chiếm dụng trên chip
Tổng số thanh ghi (Total Registers)	1,111	Bao gồm mảng đệm vòng, các thanh ghi trạng thái và FSM
Số chân giao tiếp ngoại vi (I/O Pins)	324	Kéo tải chủ yếu do bus đầu vào (32-bit) và bus đầu ra (256-bit)
Tần số hoạt động cực đại (F_{max})	54.59MHz	Tốc độ xung nhịp tối đa giới hạn bởi đường găng (Critical Path) của mạch

Bảng 5.4.1: Bảng phân tích tài nguyên sử dụng

Đánh giá chi tiết:

Tài nguyên tổ hợp (Total Combinational Functions): Hệ thống sử dụng 2,989 hàm tổ hợp (trên tổng số 3,437 Logic Elements). Đây là phần tài nguyên tiêu hao nhiều nhất. Việc gộp 2 vòng bấm (round) vào chung một chu kỳ xung nhịp buộc bộ tổng hợp (Synthesizer) phải nhân đôi các bộ cộng song song (Adder Trees), cũng như nhân đôi các trạm xử lý logic phi tuyến như Sigma (Σ), Ch, và Maj. Con số gần 3000 hàm tổ hợp này là minh chứng rõ nét cho nguyên lý thiết kế của đồ án: chấp nhận đánh đổi một lượng lớn diện tích vi mạch (Area) để triệt tiêu thời gian trễ và tối đa hóa tốc độ tính toán (Speed).

Tài nguyên tuần tự (Dedicated Logic Registers): Hệ thống sử dụng tổng cộng 1,111 thanh ghi (Registers/Flip-flops). Con số này phản ánh cực kỳ chính xác cấu trúc vi mạch RTL đã thiết kế với tỷ lệ hao hụt cực thấp. Cụ thể:

- Bộ đệm vòng (Circular Buffer) trong khối Message Expander chứa 16 từ 32-bit (chiếm 512 flip-flops).
- Các thanh ghi trạng thái làm việc từ a đến h trong khối Compression (chiếm $8 \times 32 = 256$ flip-flops).
- Các thanh ghi lưu trữ giá trị Hash trung gian từ hash_0 đến hash_7 (chiếm $8 \times 32 = 256$ flip-flops).
- Tổng cộng 3 thành phần lõi chiếm 1,024 flip-flops. Phần còn lại (87 flip-flops) được hệ thống phân bổ tối ưu cho các biến đếm (count, word_cnt), và các flip-flop trạng thái của máy FSM. Điều này cho thấy tài nguyên tuần tự được quản lý rất chặt chẽ, không có hiện tượng dư thừa.

Tín hiệu ngoại vi (I/O Pins): Thiết kế sử dụng 324 chân I/O, phục vụ cho các tín hiệu điều khiển và các bus dữ liệu song song băng thông rộng. Trong môi trường nhúng

thực tế (System-on-Chip), số lượng I/O lớn này có thể được tối ưu hóa lại bằng cách đóng gói lõi IP giao tiếp qua các chuẩn bus tuần tự chuẩn công nghiệp (như AXI4 hoặc Avalon Memory-Mapped).

Tải tín hiệu (Maximum Fan-out): Mạng lưới phân phối xung nhịp (clk) đóng vai trò là nút có mức độ phân nhánh (fan-out) cao nhất (1,111), cấp xung nhịp đồng bộ trực tiếp tới toàn bộ 1,111 thanh ghi trong hệ thống. Khả năng cấp xung đồng đều này giúp mạch tránh được các hiện tượng lệch pha trong quá trình hoạt động tốc độ cao.

Tần số hoạt động tối đa (Fmax): Với thiết kế gỡ vòng 2 bước, tín hiệu phải chạy xuyên qua 2 bộ cộng dồn, 2 bộ tính Σ , Ch, Maj trong cùng 1 chu kì clock làm tăng lượng logic tổ hợp trên đường găng (Critical Path). Vì vậy làm tăng thời gian trễ lan truyền.

Thông lượng (Throughput): $(\text{Block_in} * \text{Fmax}) / \text{Tổng số chu kì} = (512 \text{ bits} * 54.59\text{MHz}) / 50$ sẽ bằng xấp xỉ 559 Mbps.

Đánh giá công suất tiêu thụ:

PowerPlay Power Analyzer Summary	
PowerPlay Power Analyzer Status	Successful - Tue May 19 12:12:31 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	SHA256
Top-level Entity Name	top_sha256
Family	Cyclone II
Device	EP2C35F672C6
Power Models	Final
Total Thermal Power Dissipation	148.22 mW
Core Dynamic Thermal Power Dissipation	0.00 mW
Core Static Thermal Power Dissipation	80.05 mW
I/O Thermal Power Dissipation	68.17 mW
Power Estimation Confidence	Low: user provided insufficient toggle rate data

Hình 5.4.2: Công suất tiêu thụ thực tế

Loại công suất	Giá trị
Tổng công suất nhiệt phân tán (Total Thermal Power Dissipation)	148.22 mW
Công suất tĩnh lõi (Core Static Thermal Power Dissipation)	80.05 mW
Công suất tiêu tán qua các chân giao tiếp (I/O Thermal Power Dissipation)	68.17 mW
Công suất động lõi (Core Dynamic Thermal Power Dissipation)	0.00mW

Bảng 5.4.2: Bảng công suất tiêu thụ

Dựa vào thông số công suất tiêu thụ trên ta thấy được:

Công suất tĩnh (Static Power): Chiếm tỷ trọng lớn nhất với 80.05 mW (tương đương khoảng 54% tổng công suất). Đây là công suất rò rỉ của dòng chip Cyclone II bất kể mạch có đang tính toán hay không. Với một lõi IP sử dụng tới hơn 3,400 hàm logic, mức hao phí tĩnh này nằm trong phổ hoạt động bình thường của kiến trúc FPGA.

Công suất Giao tiếp (I/O Power): Đạt mức khá cao là 68.17 mW. Nguyên nhân trực tiếp đến từ đặc thù thiết kế cấp khối của mô-đun top_sha256: hệ thống sử dụng bus ngõ ra song song cực rộng (hash_result 256-bit) và bus ngõ vào 32-bit (tổng cộng 324 chân I/O). Việc kéo tải một lượng lớn các chân pin vật lý này khiến hao phí I/O tăng vọt.

Mức độ tin cậy của ước lượng Công suất động (Dynamic Power Confidence): Do hệ thống đang sử dụng phương pháp ước lượng không vector và chưa được cung cấp chuỗi dữ liệu tỷ lệ chuyển mạch thực tế (Toggle rate data), công cụ Quartus tạm thời đánh giá mức độ tin cậy ở mức thấp (Low Confidence) và ghi nhận công suất động ở mức 0.00 mW.

5.5 Kết luận và hướng phát triển trong tương lai

Kết luận:

Đồ án đã hoàn thành mục tiêu thiết kế và thực thi thành công thuật toán SHA-256 trên Kit FPGA Cyclone II. Bằng việc áp dụng kiến trúc Gỡ vòng 2 bước, thiết kế đã giảm thiểu thành công số chu kỳ hoạt động xuống còn 50 chu kỳ/khối dữ liệu. Kết hợp với bộ đệm vòng tối ưu hóa tài nguyên, hệ thống đạt được thông lượng lên tới 572.4 Mbps, đáp ứng tốt các tiêu chuẩn về gia tốc phần cứng mật mã trong khi vẫn đảm bảo tính đúng đắn về mặt chức năng.

Hướng phát triển tương lai:

Mặc dù thiết kế hiện tại đã đạt được những thông số khả quan, hệ thống vẫn còn nhiều tiềm năng để tối ưu hóa và mở rộng ứng dụng thực tiễn. Các hướng phát triển tiếp theo của đồ án bao gồm:

- **Tích hợp hệ thống SoC và Vi điều khiển:** Đóng gói lõi IP SHA-256 hiện tại bằng các chuẩn giao tiếp bus công nghiệp (như AMBA AXI4 hoặc Avalon Memory-Mapped). Việc này cho phép lõi bấm hoạt động như một bộ gia tốc bảo mật phần cứng, sẵn sàng tích hợp liền mạch vào các hệ thống nhúng.
- **Tối ưu hóa vi kiến trúc (Pipelining):** Để khắc phục điểm nghẽn về tần số hoạt động (Fmax) do đường găng của kỹ thuật Unrolling gây ra, hệ thống có thể được chèn thêm các thanh ghi đường ống (Pipeline registers) vào giữa các trạm tính toán tổ hợp. Sự kết hợp giữa Unrolling và Pipelining sẽ giúp tăng vọt tần số xung nhịp, đẩy thông lượng lên mức Gigabit/giây (Gbps)

NGUỒN THAM KHẢO

- [1] R. Martino and A. Cilardo, "SHA-2 Acceleration Meeting the Needs of Emerging Applications: A Comparative Survey," *IEEE Access*, vol. 8, pp. 28415-28436, 2020
- [2] Wang, J., Liu, G., Chen, Y. and Wang, S., 2021. Construction and analysis of SHA-256 compression function based on chaos S-box. *IEEE Access*, 9, pp.61768-61777.
- [3] Suhaili, S.b. and Watanabe, T., 2017, November. Design of high-throughput SHA-256 hash function based on FPGA. In *2017 6th International Conference on Electrical Engineering and Informatics (ICEEI)* (pp. 1-6). IEEE.
- [4] National Institute of Standards and Technology (NIST), 2002. *Secure Hash Standard (SHS)* (FIPS PUB 180-2). Washington, D.C.: U.S. Department of Commerce.
- [5] SHA-256 | COMPLETE Step-By-Step Explanation (W/ Example).
- [6] Hàm băm SHA-256 là gì? Hiểu rõ về thuật toán của SHA-256.
- [7] SHA256 – Online Tools.
- [8] Slides bài giảng Thiết kế hệ thống số với HDL.